

LLM MODEL LANDSCAPE

A Research-Backed Knowledge Base for RAG Systems

66 model and model-family cards • 15 providers • 99 ingestible records
Research cut-off: 2026-07-10

Includes capabilities, best-fit use cases, limitations, failure modes, RAG fit, pricing snapshots, deployment guidance, evaluation, governance, lifecycle management, embeddings, rerankers, OCR, safety, and voice.

Prepared as PDF, editable DOCX, and metadata-delimited TXT corpus

Important dated-snapshot notice

This document is a technical research snapshot dated 2026-07-10. Model names, aliases, pricing, quotas, regional availability, context windows, tool support, preview status, and retirement dates can change quickly. Reopen the official source links before production deployment, procurement, compliance approval, or cost forecasting.

Pricing is presented in the unit published by the provider and generally excludes taxes, currency conversion, negotiated enterprise discounts, infrastructure, vector databases, reranking, storage, egress, observability, evaluations, and engineering operations. “Provider claim” identifies positioning stated by the vendor; it is not an independent benchmark conclusion.

Contents

- Part I — Scope, model selection, taxonomy, pricing, and RAG architecture
- Part II — Provider portfolio overviews and 66 model cards
- Part III — Retrieval, chunking, evaluation, security, agents, multimodal systems, deployment, and lifecycle
- Part IV — Official source registry

Provider order: OpenAI • Anthropic • Google • Amazon • Meta • Mistral AI • Cohere • xAI • DeepSeek • Alibaba / Qwen • Microsoft • NVIDIA • IBM • Perplexity • Writer

This is a static contents guide. Use Word’s Navigation pane or PDF headings to move through sections.

Provider landscape at a glance

Provider	Cards	Primary emphasis	Deployment
OpenAI	5	frontier reasoning, agents, multimodal, realtime	managed API
Anthropic	4	long context, coding, enterprise agents	managed API / cloud
Google	5	multimodal, fast tiers, reasoning, realtime	managed API / cloud
Amazon	4	Nova family and Bedrock control plane	cloud managed
Meta	6	open-weight general, multimodal, safety	self-hosted / partners
Mistral AI	6	general, code, OCR, moderation	managed / self-hosted
Cohere	5	enterprise generation and retrieval stack	managed / private
xAI	3	reasoning, long context, voice	managed API
DeepSeek	2	low-cost long-context reasoning	managed / partner
Alibaba / Qwen	6	multilingual, coding, embedding, open weights	managed / open-weight
Microsoft	5	Phi small models and model catalog/router	Azure managed / compute
NVIDIA	4	open accelerated reasoning and vision	self-hosted / cloud
IBM	5	governed enterprise models and document AI	hybrid / managed / open
Perplexity	4	web-grounded answers and research	managed API
Writer	2	enterprise content and agent platform	managed enterprise

Part I — Foundations and selection

Purpose, scope, and research method

This knowledge base is a dated technical landscape for building and evaluating retrieval-augmented generation systems. It covers 66 model or model-family cards across 15 providers, plus retrieval, document parsing, safety, deployment, cost, evaluation, governance, and lifecycle guidance. The research cut-off is 2026-07-10.

The model market changes quickly. Names, aliases, context windows, supported regions, preview status, quotas, prices, and retirement dates can change independently. Every numerical statement should therefore be treated as a snapshot, not a permanent contract. Before procurement or production release, reopen the official URLs in the source registry and verify the exact endpoint, region, billing unit, service tier, currency, tax treatment, and license.

Research method: - Prefer first-party provider documentation, official model cards, pricing pages, cloud catalogs, and lifecycle notices. - Separate provider claims from implementation advice. Provider claims are labeled as such; failure analysis and selection guidance are analytical synthesis. - Avoid ranking models with one universal score. Real performance depends on the prompt, language, domain, retrieval corpus, tools, safety rules, context length, and infrastructure. - Treat “open-weight” as a deployment category, not a guarantee of an unrestricted open-source license. - Include non-generative components—embeddings, rerankers, OCR/document parsers, moderation, and realtime speech—because a reliable RAG system is a pipeline, not only a chat model.

How to use the three files: - The PDF is the human-readable reference and training handout. - The DOCX is editable and suitable for adding internal evaluation results, procurement notes, and organization-specific policies. - The TXT file is deliberately chunked with metadata delimiters for ingestion into a vector database or hybrid search index.

Executive selection guide

There is no universally best model. Select a portfolio by workload.

Use a frontier or strong reasoning model for difficult synthesis, coding, planning, ambiguous questions, and high-value agent decisions. Route simple tasks away from it. Use a balanced model as the default generator for enterprise chat and RAG. Use a fast or small model for query rewriting, classification, extraction, routing, guardrails, and high-volume FAQs. Use an open-weight model when data locality, customization, predictable sustained-volume economics, or sovereign deployment justifies operating the infrastructure.

Use a search-grounded model for current public information, but keep public web evidence separate from private-corpus evidence. Use a vision model when layout, images, screenshots, charts, or scanned pages carry meaning. Use OCR or a document parser to create stable text, table, and provenance records. Use embeddings for first-stage semantic retrieval, lexical search for exact identifiers, and a reranker to improve evidence ordering. Use a safety model as one defense layer, never as authorization.

A strong production pattern is a cascade: 1. Authenticate the user and derive access filters. 2. Classify the request for domain, risk, freshness, modality, and complexity. 3. Retrieve with hybrid search and mandatory metadata filters. 4. Rerank and diversify evidence. 5. Route easy questions to a cheap model and difficult questions to a stronger model. 6. Validate citations, schema, policy, and tool parameters. 7. Escalate low-confidence or high-risk results to a human. 8. Log the model version, prompt version, retrieval set, costs, latency, and outcome.

Model taxonomy and terminology

Frontier model: a provider’s strongest general model, usually with higher cost and latency. Balanced model: a production-oriented compromise among quality, speed, and price. Fast model: optimized for throughput and response time; often suitable for utility stages. Reasoning model: allocates extra inference to multi-step problems. Reasoning is not evidence and is not guaranteed correct. Small language model: a compact model suitable for low-cost, private, or narrow tasks. Open-weight model: downloadable parameters are available under a specified license. This does not automatically mean the complete training data, training code, or unrestricted commercial rights are open. Multimodal model: accepts or produces more than one modality, such as text, image, audio, or video. Context window: the maximum combined input—and sometimes output—token budget accepted by an endpoint. Advertised capacity does not guarantee uniform recall or economical use. Output limit: maximum generated tokens; it can be far smaller than the context window. Embedding model: produces vectors used for similarity search; it does not generate a natural-language answer. Reranker: scores query-document pairs after broad retrieval. OCR/document parser: extracts text, tables, reading order, coordinates, and document structure. Search-grounded model: invokes public search and cites retrieved web pages. Prompt caching: discounted or accelerated reuse of a matching prompt prefix, subject to provider-specific rules. Batch API: asynchronous or delayed processing offered at lower prices by some providers. Tool/function calling: structured requests from a model to application-defined functions. It is not permission to execute them. Structured output: generation constrained to a schema. Validation is still required.

Pricing and total-cost methodology

Token prices are not directly comparable until the billing assumptions match. A practical estimate is:

monthly model cost = (input tokens × input rate) + (cached input tokens × cached-input rate) + (output tokens × output rate) + reasoning-token charges + search/grounding/request fees + image/audio/video units + tool or hosted-compute charges + batch and service-tier adjustments.

Then add retrieval, vector database, reranking, object storage, egress, observability, evaluation, security, and engineering operations.

Example comparison method: - Build a fixed workload of at least several hundred representative requests. - Record actual tokenizer counts rather than estimating from character count. - Measure average and percentile output length; output is often much more expensive than input. - Include retries, timeouts, safety re-prompts, agent loops, and failed tool calls. - Compare quality at an agreed service-level objective, not at equal token count. - Model cached and uncached traffic separately. - For web-grounded APIs, include request/search/citation fees. - For realtime audio, use the provider's audio billing units and session-duration assumptions. - For self-hosting, calculate cost per successful request at realistic utilization, including idle capacity and redundancy.

Do not put a provider price directly into application logic. Keep a dated price catalog, alert on changes, and attach the price version to cost reports.

Reference RAG architecture

A dependable RAG pipeline has two paths: ingestion and query serving.

Ingestion: 1. Register the source with owner, classification, jurisdiction, retention, access policy, checksum, and effective date. 2. Parse the native file while preserving page numbers, headings, tables, figures, code blocks, and coordinates. 3. Apply OCR only where needed and retain confidence scores and original page images. 4. Normalize carefully; do not destroy identifiers, units, legal numbering, or table relationships. 5. Chunk by structure and meaning. Keep parent-child links and overlap only where it improves continuity. 6. Add metadata such as source_id, section_path, page, effective_from, effective_to, language, tenant, and ACL. 7. Create lexical and dense indexes. Store the embedding model and preprocessing version. 8. Run ingestion quality checks and sample visual comparisons. 9. Publish an immutable corpus version; use incremental updates and tombstones for deletions.

Query serving: 1. Authenticate and calculate the user's permissions. 2. Detect language, intent, domain, time sensitivity, risk, and required modalities. 3. Rewrite or decompose the query without losing named entities, dates, identifiers, and constraints. 4. Retrieve with dense, lexical, structured, graph, or SQL methods as appropriate. 5. Apply authorization filters before candidates reach the model. 6. Rerank and diversify; prefer authoritative and current sources. 7. Pack a bounded context with stable chunk IDs and clear delimiters. 8. Generate an answer under a policy that forbids unsupported claims. 9. Validate citations, numbers, JSON/schema, sensitive data, and tool parameters. 10. Return evidence links and log the full trace for evaluation.

The generator cannot repair evidence that was never retrieved. Measure corpus coverage, parser accuracy, retrieval recall, reranker quality, and answer grounding separately.

Part II — Provider portfolios and model cards

1. OpenAI

A broad managed-API portfolio spanning flagship reasoning, cost-optimized general models, realtime/voice, multimodal input, embeddings, moderation, and hosted tools. Strong fit for agentic applications and rapid product development; watch output-token cost, tool-specific charges, model aliases, and lifecycle changes.

Models covered

- GPT-5.6 Sol (gpt-5.6-sol (alias gpt-5.6)) — Active frontier API model; frontier; Text and image input; text output.
- GPT-5.6 Terra (gpt-5.6-terra) — Active frontier API model; balanced; Text and image input; text output.
- GPT-5.6 Luna (gpt-5.6-luna) — Active fast API model; fast; Text and image input; text output.
- GPT-Realtime 2.1 (gpt-realtime-2.1) — Active specialized model; audio; Live audio and text; conversational output.
- OpenAI Embeddings (Current OpenAI embedding family; confirm exact ID) — Active specialized family; embedding; Text to vector.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

GPT-5.6 Sol

Model ID / alias: gpt-5.6-sol (alias gpt-5.6) • **Status:** Active frontier API model • **Archetype:** frontier

Tags: frontier, reasoning, multimodal, agent, managed

Access	Managed API; selected cloud platforms
Modalities	Text and image input; text output
Advertised context	1.05M tokens
Advertised max output	128K tokens
Published pricing snapshot	\$5 input / \$30 output per 1M tokens

What it is and why it matters

OpenAI positions Sol as the highest-capability member of the GPT-5.6 family. Strong general reasoning, instruction following, coding, synthesis, and tool orchestration make it suitable as a primary generator or agent brain.

Core capabilities

- frontier reasoning and synthesis
- coding and software engineering
- function calling, web search, file search, and computer-use workflows
- multilingual and vision tasks

Best-fit use cases

- high-value agents
- complex coding
- multi-document synthesis
- hard analytical RAG

When not to choose it first

- bulk low-value classification
- deterministic transformation without validation
- latency-sensitive simple requests

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated frontier capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Frontier capability does not eliminate hallucination. It can invent citations, make invalid assumptions, over-trust retrieved text, or produce persuasive but wrong reasoning. Cost and latency rise with long prompts and extended reasoning. Cross-cutting risks

include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Route routine traffic to a cheaper sibling. Million-token prompts are costly and can dilute evidence, so retrieve selectively. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for high-value synthesis, multi-document comparison, query decomposition, and difficult answer generation after retrieval. Long context is not a replacement for retrieval, access control, citations, or freshness checks. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[OAI-MODELS] OpenAI model catalog

<https://developers.openai.com/api/docs/models> • accessed 2026-07-10

[OAI-PRICING] OpenAI API pricing

<https://developers.openai.com/api/docs/pricing> • accessed 2026-07-10

GPT-5.6 Terra

Model ID / alias: **gpt-5.6-terra** • Status: **Active frontier API model** • Archetype: **balanced**

Tags: balanced, multimodal, agent, managed

Access	Managed API
Modalities	Text and image input; text output
Advertised context	1.05M tokens
Advertised max output	128K tokens
Published pricing snapshot	\$2.50 input / \$15 output per 1M tokens

What it is and why it matters

Terra is the middle cost-performance tier in the GPT-5.6 family. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- general reasoning
- coding
- tool use
- vision
- long-context work

Best-fit use cases

- production assistants
- enterprise RAG
- coding copilots
- moderate-complexity agents

When not to choose it first

- ultra-low-cost extraction
- tasks requiring the absolute strongest reasoning tier

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Use as a primary model when Sol is too expensive. Build an evaluator and escalation policy rather than choosing from benchmark averages alone. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[OAI-MODELS] OpenAI model catalog

<https://developers.openai.com/api/docs/models> • accessed 2026-07-10

[OAI-PRICING] OpenAI API pricing

<https://developers.openai.com/api/docs/pricing> • accessed 2026-07-10

GPT-5.6 Luna

Model ID / alias: **gpt-5.6-luna** • Status: **Active fast API model** • Archetype: **fast**

Tags: fast, multimodal, managed

Access	Managed API
Modalities	Text and image input; text output
Advertised context	1.05M tokens
Advertised max output	128K tokens
Published pricing snapshot	\$1 input / \$6 output per 1M tokens

What it is and why it matters

Luna is the lowest-priced GPT-5.6 tier while retaining the family interface and long context. Designed for high throughput, low latency, and lower unit cost. It is useful for query rewriting, metadata extraction, routing, classification, and first-pass answers.

Core capabilities

- fast general generation
- classification and extraction
- query rewriting
- vision-assisted utility tasks
- tool calling

Best-fit use cases

- high-throughput RAG

- routing
- summaries
- structured extraction

When not to choose it first

- difficult proofs
- long-horizon autonomous planning
- high-stakes final answers without review

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated fast capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Fast models are more sensitive to ambiguity and retrieval noise. They can miss contradictions, omit qualifiers, or follow a plausible distractor. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Equal context limits do not imply equal reasoning quality. Use concise schemas and escalate difficult requests. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for utility stages or simple grounded Q&A. Escalate difficult and high-risk questions rather than forcing one cheap model to handle every request. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[OAI-MODELS] OpenAI model catalog

<https://developers.openai.com/api/docs/models> • accessed 2026-07-10

[OAI-PRICING] OpenAI API pricing

<https://developers.openai.com/api/docs/pricing> • accessed 2026-07-10

GPT-Realtime 2.1

Model ID / alias: gpt-realtime-2.1 • **Status:** Active specialized model • **Archetype:** audio

Tags: audio, realtime, agent

Access	Managed realtime API
Modalities	Live audio and text; conversational output
Advertised context	See live model card
Advertised max output	Streaming session output
Published pricing snapshot	Audio/text usage pricing; verify live pricing

What it is and why it matters

OpenAI lists a dedicated realtime family for speech-centric interactive applications. Supports speech input/output or realtime dialogue for voice agents, transcription, translation, and multimodal interfaces.

Core capabilities

- low-latency voice conversation

- streaming input and output
- tool-enabled voice agents
- natural turn-taking

Best-fit use cases

- voice support agents
- live tutoring
- hands-free assistants

When not to choose it first

- offline batch document analysis
- interfaces that cannot expose corrections or transcripts

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated audio capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Noise, accents, crosstalk, latency, turn-taking, numbers, names, and domain terminology remain frequent failure points. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Design for barge-in, network jitter, uncertain transcription, and confirmation of names, dates, addresses, and payments. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Transcribe and normalize the query, retrieve text evidence, then generate speech. Keep a transcript, expose citations visually, and confirm high-risk entities and numbers. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[OAI-MODELS] OpenAI model catalog

<https://developers.openai.com/api/docs/models> • accessed 2026-07-10

[OAI-PRICING] OpenAI API pricing

<https://developers.openai.com/api/docs/pricing> • accessed 2026-07-10

OpenAI Embeddings

Model ID / alias: Current OpenAI embedding family; confirm exact ID • **Status:** Active specialized family • **Archetype:** embedding

Tags: embedding, rag

Access	Managed API
Modalities	Text to vector
Advertised context	Model dependent
Advertised max output	Embedding vector
Published pricing snapshot	Model dependent; verify live pricing

What it is and why it matters

OpenAI offers dedicated embedding models separate from generative GPT models. Maps content into vectors for semantic retrieval, clustering, recommendations, deduplication, and similarity search.

Core capabilities

- semantic retrieval
- clustering
- recommendation
- similarity search

Best-fit use cases

- RAG indexes
- semantic search
- deduplication

When not to choose it first

- using cosine score as factual confidence
- replacing keyword search for codes and identifiers

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated embedding capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Embedding similarity is not truth or entailment. Scores vary by language, domain, chunk length, query style, and index configuration. Vector-only search misses exact identifiers. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Pin the exact model and dimensions. Switching embedding families normally requires re-embedding and index regression testing. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Combine dense retrieval with lexical search, metadata filters, and reranking. Evaluate recall@k and record model version and dimensions in index metadata. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[OAI-MODELS] OpenAI model catalog

<https://developers.openai.com/api/docs/models> • accessed 2026-07-10

[OAI-PRICING] OpenAI API pricing

<https://developers.openai.com/api/docs/pricing> • accessed 2026-07-10

2. Anthropic

Claude models emphasize long-context analysis, coding, enterprise assistants, tool use, and safety-oriented product design. The family is tiered from flagship to fast models. Prompt caching and batch processing can materially affect cost; extended reasoning raises latency and token usage.

Models covered

- Claude Fable 5 (See current Claude API model ID) — Active flagship; frontier; Text and image input; text output.
- Claude Opus 4.8 (See current Claude API model ID) — Active flagship; frontier; Text and image input; text output.
- Claude Sonnet 5 (See current Claude API model ID) — Active balanced flagship; balanced; Text and image input; text output.
- Claude Haiku 4.5 (See current Claude API model ID) — Active fast model; fast; Text and image input; text output.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Claude Fable 5

Model ID / alias: See current Claude API model ID • **Status:** Active flagship • **Archetype:** frontier

Tags: frontier, reasoning, agent, multimodal

Access	Anthropic API, Bedrock, Google Cloud, Microsoft Foundry where available
Modalities	Text and image input; text output
Advertised context	1M tokens
Advertised max output	128K tokens
Published pricing snapshot	\$10 input / \$50 output per 1M tokens

What it is and why it matters

Anthropic describes Fable 5 as its most capable widely released model and a next-generation model for long-running agents. Strong general reasoning, instruction following, coding, synthesis, and tool orchestration make it suitable as a primary generator or agent brain.

Core capabilities

- long-running agents
- advanced reasoning
- coding
- multimodal document work
- adaptive thinking

Best-fit use cases

- complex enterprise agents
- difficult coding
- deep synthesis
- high-value workflows

When not to choose it first

- routine high-volume traffic
- tight-latency chat
- cost-insensitive context dumping

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated frontier capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Frontier capability does not eliminate hallucination. It can invent citations, make invalid assumptions, over-trust retrieved text, or produce persuasive but wrong reasoning. Cost and latency rise with long prompts and extended reasoning. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. The premium price makes routing essential. Set

maximum reasoning/output budgets and test whether Opus or Sonnet meets the same business target. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for high-value synthesis, multi-document comparison, query decomposition, and difficult answer generation after retrieval. Long context is not a replacement for retrieval, access control, citations, or freshness checks. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[ANT-MODELS] Anthropic Claude models overview

<https://platform.claude.com/docs/en/about-claude/models/overview> • accessed 2026-07-10

[ANT-PRICING] Anthropic Claude pricing

<https://platform.claude.com/docs/en/about-claude/pricing> • accessed 2026-07-10

Claude Opus 4.8

Model ID / alias: See current Claude API model ID • Status: Active flagship • Archetype: frontier

Tags: frontier, coding, agent

Access	Anthropic API and major cloud marketplaces
Modalities	Text and image input; text output
Advertised context	1M tokens
Advertised max output	128K tokens
Published pricing snapshot	\$5 input / \$25 output per 1M tokens

What it is and why it matters

Anthropic positions Opus 4.8 for complex agentic coding and enterprise work. Strong general reasoning, instruction following, coding, synthesis, and tool orchestration make it suitable as a primary generator or agent brain.

Core capabilities

- complex agentic coding
- enterprise knowledge work
- deep analysis
- tool use

Best-fit use cases

- coding agents
- large repository analysis
- complex document comparison

When not to choose it first

- cheap batch extraction
- simple FAQ traffic

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated frontier capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Frontier capability does not eliminate hallucination. It can invent citations, make invalid assumptions, over-trust retrieved text, or produce persuasive but wrong reasoning. Cost and latency rise with long prompts and extended reasoning. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Long agent runs require checkpoints, tool authorization, spend ceilings, and deterministic verification of changes. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for high-value synthesis, multi-document comparison, query decomposition, and difficult answer generation after retrieval. Long context is not a replacement for retrieval, access control, citations, or freshness checks. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[ANT-MODELS] Anthropic Claude models overview

<https://platform.claude.com/docs/en/about-claude/models/overview> • accessed 2026-07-10

[ANT-PRICING] Anthropic Claude pricing

<https://platform.claude.com/docs/en/about-claude/pricing> • accessed 2026-07-10

Claude Sonnet 5

Model ID / alias: See current Claude API model ID • Status: Active balanced flagship • Archetype: balanced

Tags: balanced, coding, multimodal

Access	Anthropic API and major cloud marketplaces
Modalities	Text and image input; text output
Advertised context	1M tokens
Advertised max output	128K tokens
Published pricing snapshot	\$2/\$10 introductory through 2026-08-31; \$3/\$15 from 2026-09-01

What it is and why it matters

Anthropic presents Sonnet 5 as a speed-intelligence balance. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- balanced reasoning and speed
- coding
- vision
- agents
- long-context analysis

Best-fit use cases

- default enterprise assistant
- RAG generation
- coding copilots
- document automation

When not to choose it first

- ultra-cheap utility tasks
- assuming promotional pricing is permanent

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Use the post-promotion price in forecasts extending beyond August 2026 and store price effective dates. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[ANT-MODELS] Anthropic Claude models overview

<https://platform.claude.com/docs/en/about-claude/models/overview> • accessed 2026-07-10

[ANT-PRICING] Anthropic Claude pricing

<https://platform.claude.com/docs/en/about-claude/pricing> • accessed 2026-07-10

Claude Haiku 4.5

Model ID / alias: See current Claude API model ID • **Status:** Active fast model • **Archetype:** fast

Tags: fast, managed

Access	Anthropic API and cloud marketplaces
Modalities	Text and image input; text output
Advertised context	200K tokens
Advertised max output	64K tokens
Published pricing snapshot	\$1 input / \$5 output per 1M tokens

What it is and why it matters

Anthropic describes Haiku 4.5 as its fastest model with near-frontier capability. Designed for high throughput, low latency, and lower unit cost. It is useful for query rewriting, metadata extraction, routing, classification, and first-pass answers.

Core capabilities

- low-latency generation
- classification
- extraction
- simple tool use

- utility work

Best-fit use cases

- query classification
- RAG routing
- summarization
- high-volume chat

When not to choose it first

- the hardest reasoning and coding
- very large context workflows

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated fast capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Fast models are more sensitive to ambiguity and retrieval noise. They can miss contradictions, omit qualifiers, or follow a plausible distractor. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Use schemas and evaluation sets. Cheap extraction errors can contaminate metadata, routing, and later retrieval. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for utility stages or simple grounded Q&A. Escalate difficult and high-risk questions rather than forcing one cheap model to handle every request. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[ANT-MODELS] Anthropic Claude models overview

<https://platform.claude.com/docs/en/about-claude/models/overview> • accessed 2026-07-10

[ANT-PRICING] Anthropic Claude pricing

<https://platform.claude.com/docs/en/about-claude/pricing> • accessed 2026-07-10

3. Google

Gemini provides multimodal, long-context, fast, reasoning, and realtime tiers, integrated through the Gemini API and Google Cloud. Pricing often changes by context length, modality, batch use, grounding, or cached input. Preview model stability and regional availability require attention.

Models covered

- Gemini 3.5 Flash (gemini-3.5-flash) — Active production model; balanced; Text, image, video, and audio inputs; text output.
- Gemini 3.1 Pro Preview (gemini-3.1-pro-preview) — Preview; frontier; Multimodal input; text output.
- Gemini 3.1 Flash-Lite (gemini-3.1-flash-lite) — Active efficiency model; fast; Text/image/video input; audio priced separately; text output.
- Gemini 2.5 Pro (gemini-2.5-pro) — Active established model; reasoning; Multimodal input; text output.
- Gemini 3.1 Flash Live Preview (gemini-3.1-flash-live-preview) — Preview realtime model; audio; Audio, text, image/video input; audio/text output.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Gemini 3.5 Flash

Model ID / alias: gemini-3.5-flash • **Status:** Active production model • **Archetype:** balanced

Tags: balanced, multimodal, search, agent

Access	Gemini Developer API and Google Cloud
Modalities	Text, image, video, and audio inputs; text output
Advertised context	See live model page
Advertised max output	See live model page
Published pricing snapshot	\$1.50 input / \$9 output; batch \$0.75/\$4.50 per 1M tokens

What it is and why it matters

Google calls Gemini 3.5 Flash its most intelligent model built for speed and highlights search and grounding. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- frontier intelligence at speed
- search and grounding
- multimodal understanding
- agentic workflows

Best-fit use cases

- grounded assistants
- multimodal RAG
- production agents
- high-quality chat

When not to choose it first

- ignoring per-search grounding charges
- assuming free-tier data terms equal paid tier

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Grounding can issue multiple paid search queries per request. Instrument query counts

separately from tokens. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[GOO-35FLASH] Gemini 3.5 Flash

<https://ai.google.dev/gemini-api/docs/models/gemini-3.5-flash> • accessed 2026-07-10

[GOO-PRICING] Google Gemini Developer API pricing and model list

<https://ai.google.dev/gemini-api/docs/pricing> • accessed 2026-07-10

Gemini 3.1 Pro Preview

Model ID / alias: **gemini-3.1-pro-preview** • Status: **Preview** • Archetype: **frontier**

Tags: preview, frontier, multimodal, agent

Access	Gemini Developer API; preview terms apply
Modalities	Multimodal input; text output
Advertised context	Large context; verify model page
Advertised max output	See live model page
Published pricing snapshot	\$2/\$12 for prompts ≤200K; \$4/\$18 above 200K

What it is and why it matters

Google describes the preview as a major performance and usability update for multimodal and agentic work. Strong general reasoning, instruction following, coding, synthesis, and tool orchestration make it suitable as a primary generator or agent brain.

Core capabilities

- advanced multimodal understanding
- agentic capability
- software engineering
- custom tools
- long-context reasoning

Best-fit use cases

- advanced agent prototypes
- large code/document analysis
- complex multimodal tasks

When not to choose it first

- production dependency without fallback
- unbounded prompts above the 200K price threshold

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated frontier capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Frontier capability does not eliminate hallucination. It can invent citations, make invalid assumptions, over-trust retrieved text, or produce persuasive but wrong reasoning. Cost and latency rise with long prompts and extended reasoning. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Preview IDs and behavior can change. Maintain contract tests and a stable fallback. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for high-value synthesis, multi-document comparison, query decomposition, and difficult answer generation after retrieval. Long context is not a replacement for retrieval, access control, citations, or freshness checks. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[GOO-31PRO] Gemini 3.1 Pro Preview

<https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview> • accessed 2026-07-10

[GOO-PRICING] Google Gemini Developer API pricing and model list

<https://ai.google.dev/gemini-api/docs/pricing> • accessed 2026-07-10

Gemini 3.1 Flash-Lite

Model ID / alias: **gemini-3.1-flash-lite** • Status: **Active efficiency model** • Archetype: **fast**

Tags: fast, multimodal, managed

Access	Gemini Developer API
Modalities	Text/image/video input; audio priced separately; text output
Advertised context	See live model page
Advertised max output	See live model page
Published pricing snapshot	\$0.25 input (\$0.50 audio) / \$1.50 output; batch \$0.125/\$0.75

What it is and why it matters

Google calls it its most cost-efficient model for high-volume agentic tasks, translation, and simple data processing. Designed for high throughput, low latency, and lower unit cost. It is useful for query rewriting, metadata extraction, routing, classification, and first-pass answers.

Core capabilities

- high-volume agent utilities
- translation
- simple data processing
- multimodal extraction

Best-fit use cases

- classification
- translation
- query rewriting
- bulk extraction

When not to choose it first

- hard reasoning
- difficult code generation
- large noisy evidence sets

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated fast capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Fast models are more sensitive to ambiguity and retrieval noise. They can miss contradictions, omit qualifiers, or follow a plausible distractor. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Use it for pipeline stages rather than automatically for the final answer. Evaluate semantic omissions and numeric accuracy. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for utility stages or simple grounded Q&A. Escalate difficult and high-risk questions rather than forcing one cheap model to handle every request. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[GOO-PRICING] Google Gemini Developer API pricing and model list
<https://ai.google.dev/gemini-api/docs/pricing> • accessed 2026-07-10

Gemini 2.5 Pro

Model ID / alias: gemini-2.5-pro • Status: Active established model • Archetype: reasoning

Tags: reasoning, multimodal

Access	Gemini API and Vertex AI
Modalities	Multimodal input; text output
Advertised context	Large context; verify model page
Advertised max output	Model dependent
Published pricing snapshot	See current pricing page; tiered pricing may apply

What it is and why it matters

Google positions Gemini 2.5 Pro for complex reasoning, code, mathematics, STEM, and large bodies of data. Optimized for deliberate multi-step reasoning, planning, coding, mathematics, and hard analytical tasks where extra inference work can improve reliability.

Core capabilities

- complex reasoning
- code and mathematics
- STEM
- large datasets and codebases
- document analysis

Best-fit use cases

- complex analysis
- codebase review
- scientific reasoning
- long-document work

When not to choose it first

- simple high-throughput tasks
- selecting only because it is familiar without comparing current models

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated reasoning capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Reasoning tokens increase cost and latency and can still produce a confident wrong path. Internal reasoning is not auditable evidence. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. It is a useful stable comparison baseline for newer previews. Test exact API version and safety settings. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for evidence reconciliation, retrieval planning, and analytical answers. Require source-linked conclusions and use lower-cost models for easy requests. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[GOO-25PRO] Gemini 2.5 Pro
<https://ai.google.dev/gemini-api/docs/models/gemini-2.5-pro> • accessed 2026-07-10

[GOO-PRICING] Google Gemini Developer API pricing and model list
<https://ai.google.dev/gemini-api/docs/pricing> • accessed 2026-07-10

Gemini 3.1 Flash Live Preview

Model ID / alias: **gemini-3.1-flash-live-preview** • Status: **Preview realtime model** • Archetype: **audio**

Tags: audio, realtime, preview

Access	Gemini live API
Modalities	Audio, text, image/video input; audio/text output
Advertised context	Session oriented
Advertised max output	Streaming
Published pricing snapshot	\$0.75 text input; \$3 audio input; \$4.50 text output; \$12 audio output per 1M tokens

What it is and why it matters

Google describes it as a low-latency audio-to-audio model optimized for realtime dialogue. Supports speech input/output or realtime dialogue for voice agents, transcription, translation, and multimodal interfaces.

Core capabilities

- audio-to-audio dialogue
- acoustic nuance detection
- multimodal awareness
- realtime grounding

Best-fit use cases

- voice agents
- live assistants
- multimodal customer support

When not to choose it first

- production without preview fallback
- silent handling of uncertain numbers and names

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated audio capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Noise, accents, crosstalk, latency, turn-taking, numbers, names, and domain terminology remain frequent failure points. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Keep a machine-readable transcript and tool trace. Voice-only audit trails are insufficient for regulated or transactional workflows. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Transcribe and normalize the query, retrieve text evidence, then generate speech. Keep a transcript, expose citations visually, and confirm high-risk entities and numbers. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[GOO-PRICING] Google Gemini Developer API pricing and model list

<https://ai.google.dev/gemini-api/docs/pricing> • accessed 2026-07-10

4. Amazon

Amazon Nova is a first-party Bedrock family covering text/vision, speech, and multiple capability tiers. Bedrock also acts as a multi-provider control plane. Features can differ by region and model, and legacy Nova releases have explicit lifecycle dates.

Models covered

- Amazon Nova 2 Lite (Bedrock model ID varies by region/profile) — Active; balanced; Text, image, and video input; text output.
- Amazon Nova 2 Sonic (Amazon Nova 2 Sonic) — Active realtime model; audio; Speech/text streaming input and output.
- Amazon Nova Premier (Amazon Nova Premier) — Legacy; model card lists end of life 2026-09-14; frontier; Multimodal input; text output.
- Amazon Nova Micro (Amazon Nova Micro) — Active text efficiency model; fast; Text input/output.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Amazon Nova 2 Lite

Model ID / alias: Bedrock model ID varies by region/profile • **Status:** Active • **Archetype:** balanced

Tags: aws, multimodal, managed

Access	Amazon Bedrock
Modalities	Text, image, and video input; text output
Advertised context	1M tokens
Advertised max output	64K tokens
Published pricing snapshot	Region and service-tier dependent

What it is and why it matters

The AWS model card lists 1M context, 64K output, multimodal input, tool calling, and prompt caching. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- multimodal understanding
- tool calling
- prompt caching
- long-context analysis

Best-fit use cases

- AWS-native multimodal assistants
- Bedrock applications
- video and document analysis

When not to choose it first

- assuming every Bedrock feature supports the model
- cross-region deployment without checks

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. The card states that certain Bedrock capabilities such as Knowledge Bases, Agents, or structured outputs are not supported on some interfaces. Verify exact endpoint and region. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[AWS-NOVA2L] Amazon Nova 2 Lite model card

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-2-lite.html> • accessed 2026-07-10

[AWS-BEDROCK] Amazon Bedrock pricing

<https://aws.amazon.com/bedrock/pricing/> • accessed 2026-07-10

[AWS-MODELS] Amazon Bedrock supported foundation models

<https://docs.aws.amazon.com/bedrock/latest/userguide/models.html> • accessed 2026-07-10

Amazon Nova 2 Sonic

Model ID / alias: Amazon Nova 2 Sonic • Status: Active realtime model • Archetype: audio

Tags: audio, aws, realtime

Access	Amazon Bedrock bidirectional streaming
Modalities	Speech/text streaming input and output
Advertised context	1M tokens
Advertised max output	64K tokens
Published pricing snapshot	Region and usage dependent

What it is and why it matters

AWS describes Nova 2 Sonic as a realtime speech-to-speech model with bidirectional streaming. Supports speech input/output or realtime dialogue for voice agents, transcription, translation, and multimodal interfaces.

Core capabilities

- realtime speech-to-speech
- bidirectional streaming
- voice agents
- long-session context

Best-fit use cases

- AWS voice assistants
- contact centers
- interactive spoken agents

When not to choose it first

- features not supported by its streaming endpoint
- workflows without interruption handling

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated audio capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Noise, accents, crosstalk, latency, turn-taking, numbers, names, and domain terminology remain frequent failure points. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Bedrock feature support differs from text models. Treat endpoint capabilities, session limits, and tool integration as separate acceptance criteria. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Transcribe and normalize the query, retrieve text evidence, then generate speech. Keep a transcript, expose citations visually, and confirm high-risk entities and numbers. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[AWS-NOVA2S] Amazon Nova 2 Sonic model card

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-2-sonic.html> • accessed 2026-07-10

[AWS-BEDROCK] Amazon Bedrock pricing

<https://aws.amazon.com/bedrock/pricing/> • accessed 2026-07-10

Amazon Nova Premier

Model ID / alias: Amazon Nova Premier • **Status:** Legacy; model card lists end of life 2026-09-14 • **Archetype:** frontier

Tags: legacy, aws, multimodal

Access	Amazon Bedrock
Modalities	Multimodal input; text output
Advertised context	1M tokens
Advertised max output	25K tokens
Published pricing snapshot	Region dependent

What it is and why it matters

AWS positioned Premier for complex reasoning, agents, and distillation. Strong general reasoning, instruction following, coding, synthesis, and tool orchestration make it suitable as a primary generator or agent brain.

Core capabilities

- complex reasoning
- agentic tasks
- teacher model for distillation
- multimodal analysis

Best-fit use cases

- existing workloads
- migration benchmarking
- limited distillation experiments before retirement

When not to choose it first

- new long-lived production designs
- ignoring published end-of-life date

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated frontier capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Frontier capability does not eliminate hallucination. It can invent citations, make invalid assumptions, over-trust retrieved text, or produce persuasive but wrong reasoning. Cost and latency rise with long prompts and extended reasoning. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. A technically capable model can still be the wrong dependency. Build a replacement benchmark and migrate before 2026-09-14. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for high-value synthesis, multi-document comparison, query decomposition, and difficult answer generation after retrieval. Long context is not a replacement for retrieval, access control, citations, or freshness checks. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[AWS-NOVAP] Amazon Nova Premier model card

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-premier.html> • accessed 2026-07-10

[AWS-BEDROCK] Amazon Bedrock pricing

<https://aws.amazon.com/bedrock/pricing/> • accessed 2026-07-10

Amazon Nova Micro

Model ID / alias: Amazon Nova Micro • Status: Active text efficiency model • Archetype: fast

Tags: fast, aws, text

Access	Amazon Bedrock
Modalities	Text input/output
Advertised context	128K tokens
Advertised max output	5K tokens
Published pricing snapshot	Region dependent

What it is and why it matters

AWS positions Nova Micro as a fast, text-only member of the Nova family. Designed for high throughput, low latency, and lower unit cost. It is useful for query rewriting, metadata extraction, routing, classification, and first-pass answers.

Core capabilities

- fast text generation
- classification
- summarization
- low-cost text workloads

Best-fit use cases

- routing

- text extraction
- simple assistants

When not to choose it first

- vision tasks
- long-form output beyond limit
- hard reasoning

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated fast capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Fast models are more sensitive to ambiguity and retrieval noise. They can miss contradictions, omit qualifiers, or follow a plausible distractor. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Treat the 5K output limit as a product constraint and use structured short outputs. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for utility stages or simple grounded Q&A. Escalate difficult and high-risk questions rather than forcing one cheap model to handle every request. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[AWS-BEDROCK] Amazon Bedrock pricing

<https://aws.amazon.com/bedrock/pricing/> • accessed 2026-07-10

[AWS-MODELS] Amazon Bedrock supported foundation models

<https://docs.aws.amazon.com/bedrock/latest/userguide/models.html> • accessed 2026-07-10

5. Meta

Llama is a major open-weight model ecosystem with general, multimodal, small, and safety models. It offers deployment control and a large tooling community. The deployer remains responsible for infrastructure, safety, licensing, evaluation, and model-specific prompt formatting.

Models covered

- Llama 4 Maverick (Llama 4 Maverick) — Open-weight active family; open; Natively multimodal.
- Llama 4 Scout (Llama 4 Scout) — Open-weight active family; open; Natively multimodal.
- Llama 4 Behemoth (Llama 4 Behemoth) — Preview/research status; confirm before use; frontier; Multimodal family.
- Llama 3.3 70B Instruct (Llama 3.3 70B Instruct) — Established open-weight model; open; Text.
- Llama 3.2 1B/3B (Llama 3.2 1B and 3B) — Established small open-weight family; small; Text.
- Llama Guard 4 (Llama Guard 4) — Active safeguard model; safety; Text and multimodal safety classification where supported.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Llama 4 Maverick

Model ID / alias: Llama 4 Maverick • **Status:** Open-weight active family • **Archetype:** open

Tags: open, multimodal, moe

Access	Self-hosting and partner clouds subject to license
Modalities	Natively multimodal
Advertised context	See official model card
Advertised max output	Deployment dependent
Published pricing snapshot	No universal API price; infrastructure or partner price applies

What it is and why it matters

Meta describes Maverick as a natively multimodal mixture-of-experts model with 17B active parameters and 128 experts. Open-weight access provides control over hosting, quantization, fine-tuning, data locality, observability, and economics at sustained volume.

Core capabilities

- multimodal chat
- general reasoning
- MoE efficiency
- customization and fine-tuning

Best-fit use cases

- private RAG
- custom enterprise assistants
- open multimodal systems

When not to choose it first

- teams without inference operations
- assuming open-weight means unrestricted use
- copying hosted prompts to quantized deployment without testing

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated open capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

The deployer owns GPU capacity, serving optimization, upgrades, monitoring, abuse prevention, safety, and license compliance. Downloadable weights do not imply low total cost. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Serving quality depends on exact weights, quantization, engine, tensor parallelism, and prompt

format. Record all of them in evaluation reports. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

A strong fit for private or regulated RAG when the organization can operate the stack. Benchmark the exact quantization, engine, prompt format, and hardware. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[META-L4] Meta Llama 4 announcement

<https://ai.meta.com/blog/llama-4-multimodal-intelligence/> • accessed 2026-07-10

[META-L4CARD] Meta Llama 4 model card

<https://www.llama.com/docs/model-cards-and-prompt-formats/llama4/> • accessed 2026-07-10

[META-COOKBOOK] Meta Llama Cookbook

<https://github.com/meta-llama/llama-cookbook> • accessed 2026-07-10

Llama 4 Scout

Model ID / alias: Llama 4 Scout • **Status:** Open-weight active family • **Archetype:** open

Tags: open, multimodal, long-context

Access	Self-hosting and partner clouds
Modalities	Natively multimodal
Advertised context	Long-context family; verify model card
Advertised max output	Deployment dependent
Published pricing snapshot	Self-hosting or partner dependent

What it is and why it matters

Meta describes Scout as a 17B-active, 16-expert model that can fit on a single H100 with Int4 quantization. Open-weight access provides control over hosting, quantization, fine-tuning, data locality, observability, and economics at sustained volume.

Core capabilities

- multimodal understanding
- long-context workflows
- efficient MoE inference
- customization

Best-fit use cases

- private long-context RAG
- document assistants
- research prototypes

When not to choose it first

- treating single-GPU fit as a production capacity plan
- unverified extreme-context retrieval

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated open capabilities, its available deployment route, and the possibility of comparing it with cheaper or

stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

The deployer owns GPU capacity, serving optimization, upgrades, monitoring, abuse prevention, safety, and license compliance. Downloadable weights do not imply low total cost. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Weight fit is not a throughput or latency guarantee. Benchmark concurrency, KV-cache growth, and retrieval length. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

A strong fit for private or regulated RAG when the organization can operate the stack. Benchmark the exact quantization, engine, prompt format, and hardware. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[META-L4] Meta Llama 4 announcement

<https://ai.meta.com/blog/llama-4-multimodal-intelligence/> • accessed 2026-07-10

[META-L4CARD] Meta Llama 4 model card

<https://www.llama.com/docs/model-cards-and-prompt-formats/llama4/> • accessed 2026-07-10

Llama 4 Behemoth

Model ID / alias: Llama 4 Behemoth • **Status:** Preview/research status; confirm before use • **Archetype:** frontier

Tags: preview, open, research

Access	Not treated as a broadly deployed stable dependency in the announcement
Modalities	Multimodal family
Advertised context	Not stable in this snapshot
Advertised max output	Not stable in this snapshot
Published pricing snapshot	Not standardized

What it is and why it matters

Meta introduced Behemoth as a larger model still in training in the Llama 4 announcement. Strong general reasoning, instruction following, coding, synthesis, and tool orchestration make it suitable as a primary generator or agent brain.

Core capabilities

- teacher-scale reasoning
- research and distillation potential

Best-fit use cases

- tracking frontier open-model research

When not to choose it first

- production commitments based only on an announcement

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated frontier capabilities, its available deployment route, and the possibility of comparing it with cheaper or

stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Frontier capability does not eliminate hallucination. It can invent citations, make invalid assumptions, over-trust retrieved text, or produce persuasive but wrong reasoning. Cost and latency rise with long prompts and extended reasoning. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Exclude announced-but-not-stable models from routing until model card, license, artifacts, and serving path are verified. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for high-value synthesis, multi-document comparison, query decomposition, and difficult answer generation after retrieval. Long context is not a replacement for retrieval, access control, citations, or freshness checks. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[META-L4] Meta Llama 4 announcement

<https://ai.meta.com/blog/llama-4-multimodal-intelligence/> • accessed 2026-07-10

Llama 3.3 70B Instruct

Model ID / alias: Llama 3.3 70B Instruct • **Status:** Established open-weight model • **Archetype:** open

Tags: open, text, established

Access	Self-hosting and inference providers
Modalities	Text
Advertised context	See endpoint/model card
Advertised max output	Provider dependent
Published pricing snapshot	Infrastructure or partner dependent

What it is and why it matters

Meta described Llama 3.3 70B as reaching performance similar to Llama 3.1 405B on selected evaluations at lower serving cost. Open-weight access provides control over hosting, quantization, fine-tuning, data locality, observability, and economics at sustained volume.

Core capabilities

- general text generation
- coding
- multilingual assistance
- mature fine-tuning ecosystem

Best-fit use cases

- stable open text RAG
- cost-controlled private deployment
- fine-tuning

When not to choose it first

- native image understanding

- assuming it matches newer multimodal families

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated open capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

The deployer owns GPU capacity, serving optimization, upgrades, monitoring, abuse prevention, safety, and license compliance. Downloadable weights do not imply low total cost. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. It remains useful where ecosystem maturity matters. Compare it with Llama 4 and current smaller models on your domain. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

A strong fit for private or regulated RAG when the organization can operate the stack. Benchmark the exact quantization, engine, prompt format, and hardware. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[META-COOKBOOK] Meta Llama Cookbook

<https://github.com/meta-llama/llama-cookbook> • accessed 2026-07-10

Llama 3.2 1B/3B

Model ID / alias: Llama 3.2 1B and 3B • Status: Established small open-weight family • Archetype: small

Tags: small, open, edge

Access	On-device, edge, self-hosted, and partner runtimes
Modalities	Text
Advertised context	Model-card dependent
Advertised max output	Runtime dependent
Published pricing snapshot	Device or infrastructure dependent

What it is and why it matters

Meta introduced the 1B and 3B variants for mobile and edge scenarios. Small language models can run cheaply, privately, and sometimes at the edge. They are effective for narrow extraction, routing, local assistants, and constrained domain tasks.

Core capabilities

- on-device assistance
- lightweight extraction
- local summarization
- privacy-sensitive workflows

Best-fit use cases

- edge assistants
- offline routing
- narrow domain tasks

When not to choose it first

- open-ended expert reasoning
- large noisy RAG contexts
- high-stakes autonomous decisions

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated small capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

They have less world knowledge and reasoning headroom and are more likely to fail on compound instructions, rare concepts, multilingual edge cases, and conflicting evidence. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Quantization and device runtime can change output quality. Treat the deployed artifact as the model under test. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Keep the task narrow, retrieve concise evidence, use schemas, and validate outputs. A small model plus excellent retrieval can excel on a narrow domain but not every open-ended query. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[META-L32] Meta Llama 3.2 announcement
<https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/> • accessed 2026-07-10

Llama Guard 4

Model ID / alias: Llama Guard 4 • Status: Active safeguard model • Archetype: safety

Tags: safety, open, guardrail

Access	Open-weight or partner deployment subject to license
Modalities	Text and multimodal safety classification where supported
Advertised context	See model card
Advertised max output	Safety labels/explanations
Published pricing snapshot	Infrastructure or partner dependent

What it is and why it matters

Meta presents Llama Guard 4 as a 12B safeguard model. Classifies or moderates prompts and responses to support policy enforcement and defense in depth.

Core capabilities

- prompt classification
- response classification
- policy enforcement support
- multilingual safety workflows

Best-fit use cases

- defense-in-depth moderation
- agent guardrails
- RAG input and output screening

When not to choose it first

- using it as authorization
- assuming default taxonomy matches company policy

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated safety capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

No classifier is complete. False positives block legitimate work; false negatives allow harmful or sensitive content. Adversarial wording and multilingual nuance require testing. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Map its categories to company policy, test false positives and false negatives, and keep human escalation. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Apply safety checks before retrieval, before tools, and after generation. Add authorization and DLP; a content classifier is not an access-control system. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[META-GUARD4] Meta Llama Guard 4 model card

<https://www.llama.com/docs/model-cards-and-prompt-formats/llama-guard-4/> • accessed 2026-07-10

6. Mistral AI

Mistral offers commercial and open-weight general models plus specialized coding, OCR, moderation, and embedding capabilities. The catalog is attractive for European deployment, portability, and efficient models. Exact access method and pricing vary across Mistral and partner clouds.

Models covered

- Mistral Medium 3.5 (Current dated API identifier) — Active featured model; balanced; Multimodal.
- Mistral Small 4 (Current dated API identifier) — Active featured model; balanced; Text and supported multimodal inputs by endpoint.
- Mistral Large 3 (Mistral Large 3) — Active open general-purpose family; open; Multimodal.
- Devstral 2 (Devstral 2) — Active coding model; reasoning; Text/code.
- Mistral OCR 4 (Mistral OCR 4) — Active specialized model; ocr; Document/image to structured text.
- Mistral Moderation 2 (Mistral Moderation 2) — Active specialized model; safety; Text classification.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Mistral Medium 3.5

Model ID / alias: Current dated API identifier • **Status:** Active featured model • **Archetype:** balanced

Tags: balanced, multimodal, agent

Access	Mistral API, cloud partners, and available deployment options
Modalities	Multimodal
Advertised context	See current catalog
Advertised max output	See current catalog
Published pricing snapshot	Endpoint dependent

What it is and why it matters

Mistral lists Medium 3.5 as a frontier-class multimodal model for agentic and coding work. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- agentic workflows
- coding
- multimodal understanding
- enterprise tasks

Best-fit use cases

- enterprise RAG
- coding assistants
- multimodal agents

When not to choose it first

- assuming one price across direct API and marketplaces

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Mistral maintains an active lifecycle table. Pin a dated identifier instead of only a family name. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[MISTRAL-MODELS] **Mistral models overview and lifecycle**
<https://docs.mistral.ai/models/overview> • accessed 2026-07-10

[AWS-BEDROCK] **Amazon Bedrock pricing**
<https://aws.amazon.com/bedrock/pricing/> • accessed 2026-07-10

Mistral Small 4

Model ID / alias: Current dated API identifier • **Status:** Active featured model • **Archetype:** balanced

Tags: balanced, reasoning, coding

Access	Mistral and partner endpoints
Modalities	Text and supported multimodal inputs by endpoint
Advertised context	See catalog
Advertised max output	See catalog
Published pricing snapshot	Provider dependent

What it is and why it matters

Mistral describes Small 4 as a hybrid instruct, reasoning, and coding model focused on efficiency. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- hybrid instruction following
- reasoning
- coding
- efficient deployment

Best-fit use cases

- cost-aware agents
- RAG generation
- coding utilities

When not to choose it first

- assuming small means edge-sized
- hardest frontier tasks without comparison

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations,

instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Evaluate standard and reasoning modes separately because token behavior, cost, and latency can differ. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[MISTRAL-MODELS] Mistral models overview and lifecycle
<https://docs.mistral.ai/models/overview> • accessed 2026-07-10

Mistral Large 3

Model ID / alias: Mistral Large 3 • Status: Active open general-purpose family • Archetype: open

Tags: open, multimodal

Access	Open deployment and partner endpoints subject to license
Modalities	Multimodal
Advertised context	See model card
Advertised max output	Deployment dependent
Published pricing snapshot	Example Bedrock US price about \$0.50 input / \$1.50 output per 1M tokens

What it is and why it matters

Mistral lists Large 3 as an open general-purpose multimodal model. Open-weight access provides control over hosting, quantization, fine-tuning, data locality, observability, and economics at sustained volume.

Core capabilities

- general multimodal work
- enterprise customization
- open deployment
- tool use

Best-fit use cases

- cost-conscious open multimodal RAG
- private deployment
- custom agents

When not to choose it first

- using a marketplace price as universal
- operating without model monitoring

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated open capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

The deployer owns GPU capacity, serving optimization, upgrades, monitoring, abuse prevention, safety, and license compliance. Downloadable weights do not imply low total cost. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Data region, SLA, throughput tier, and version availability may matter more than nominal token price. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

A strong fit for private or regulated RAG when the organization can operate the stack. Benchmark the exact quantization, engine, prompt format, and hardware. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[MISTRAL-MODELS] Mistral models overview and lifecycle
<https://docs.mistral.ai/models/overview> • accessed 2026-07-10

[AWS-BEDROCK] Amazon Bedrock pricing
<https://aws.amazon.com/bedrock/pricing/> • accessed 2026-07-10

Devstral 2

Model ID / alias: Devstral 2 • Status: Active coding model • Archetype: reasoning

Tags: coding, agent, reasoning

Access	Mistral and partner deployments
Modalities	Text/code
Advertised context	See current model page
Advertised max output	See current model page
Published pricing snapshot	Example Bedrock US price about \$0.40 input / \$2 output per 1M tokens

What it is and why it matters

Mistral positions Devstral 2 as a frontier code-agent model. Optimized for deliberate multi-step reasoning, planning, coding, mathematics, and hard analytical tasks where extra inference work can improve reliability.

Core capabilities

- software-engineering agents
- repository navigation
- code generation
- tool use

Best-fit use cases

- coding agents
- issue resolution
- codebase Q&A

When not to choose it first

- general business chat where a balanced model is cheaper
- unreviewed code changes

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated reasoning capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Reasoning tokens increase cost and latency and can still produce a confident wrong path. Internal reasoning is not auditable evidence. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Evaluate patch correctness, test pass rate, security regressions, and repository-scale navigation. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for evidence reconciliation, retrieval planning, and analytical answers. Require source-linked conclusions and use lower-cost models for easy requests. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[MISTRAL-MODELS] Mistral models overview and lifecycle

<https://docs.mistral.ai/models/overview> • accessed 2026-07-10

[AWS-BEDROCK] Amazon Bedrock pricing

<https://aws.amazon.com/bedrock/pricing/> • accessed 2026-07-10

Mistral OCR 4

Model ID / alias: Mistral OCR 4 • **Status:** Active specialized model • **Archetype:** ocr

Tags: ocr, document, rag

Access	Mistral document/OCR API and supported platforms
Modalities	Document/image to structured text
Advertised context	Document service limits apply
Advertised max output	Structured text/layout
Published pricing snapshot	Page or document pricing; verify live endpoint

What it is and why it matters

Mistral lists OCR 4 as a specialized OCR/document model. Converts visually rich documents into structured text or layout representations for indexing scans, tables, slides, and forms.

Core capabilities

- document parsing
- OCR
- layout understanding
- table and page extraction

Best-fit use cases

- RAG ingestion
- scanned PDFs
- slide and form extraction

When not to choose it first

- discarding original page images
- assuming perfect table reconstruction

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated ocr capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

OCR errors concentrate in small fonts, handwriting, merged cells, formulas, multi-column pages, and degraded scans. Structure recovery can be approximate. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Run parser regression tests and store parser version and page coordinates so answers can link to evidence. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Store the raw file, page image, extracted text, coordinates, confidence, and parser version. Support visual verification when confidence is low. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[MISTRAL-MODELS] Mistral models overview and lifecycle
<https://docs.mistral.ai/models/overview> • accessed 2026-07-10

Mistral Moderation 2

Model ID / alias: Mistral Moderation 2 • **Status:** Active specialized model • **Archetype:** safety

Tags: safety, moderation

Access	Mistral API
Modalities	Text classification
Advertised context	Endpoint dependent
Advertised max output	Moderation categories/scores
Published pricing snapshot	Verify current pricing

What it is and why it matters

Mistral lists Moderation 2 as its current moderation family. Classifies or moderates prompts and responses to support policy enforcement and defense in depth.

Core capabilities

- content moderation
- prompt and response screening
- policy categorization

Best-fit use cases

- guardrails
- user-generated-content screening
- agent safety checks

When not to choose it first

- treating moderation as authorization
- deployment without policy calibration

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated safety capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

No classifier is complete. False positives block legitimate work; false negatives allow harmful or sensitive content. Adversarial wording and multilingual nuance require testing. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Calibrate thresholds by harm, language, and user type and log the policy version used for decisions. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Apply safety checks before retrieval, before tools, and after generation. Add authorization and DLP; a content classifier is not an access-control system. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[MISTRAL-MODELS] Mistral models overview and lifecycle

<https://docs.mistral.ai/models/overview> • accessed 2026-07-10

7. Cohere

Cohere focuses on enterprise generation and retrieval. Command models support business agents and RAG, while Embed and Rerank are dedicated retrieval components. This end-to-end retrieval stack is useful when evidence ranking matters as much as generation.

Models covered

- Command A+ (command-a-plus-05-2026) — Active newest Command family; frontier; Text and vision.
- Command A (command-a-03-2025 or current stable ID) — Active enterprise model; balanced; Text.
- Command A Reasoning (Current Command A Reasoning ID) — Active reasoning model; reasoning; Text.
- Embed v4.0 (embed-v4.0) — Active embedding model; embedding; Text, images, and mixed-content documents.
- Rerank v4 Pro (Current rerank-v4-pro ID) — Active reranker; reranker; Query and document pairs.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Command A+

Model ID / alias: command-a-plus-05-2026 • **Status:** Active newest Command family • **Archetype:** frontier

Tags: frontier, enterprise, vision, moe

Access	Cohere API and Model Vault options
Modalities	Text and vision
Advertised context	128K tokens
Advertised max output	64K tokens
Published pricing snapshot	Free within launch rate limits; production terms can differ

What it is and why it matters

Cohere describes A+ as its first mixture-of-experts Command A model and states it fits one B200 or two H100 GPUs. Strong general reasoning, instruction following, coding, synthesis, and tool orchestration make it suitable as a primary generator or agent brain.

Core capabilities

- agentic reasoning
- vision
- translation
- tool use
- enterprise deployment

Best-fit use cases

- enterprise agents
- multilingual assistants
- vision-enhanced knowledge work

When not to choose it first

- assuming launch free tier defines production cost
- deployment without hardware and rate-limit checks

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated frontier capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Frontier capability does not eliminate hallucination. It can invent citations, make invalid assumptions, over-trust retrieved text, or produce persuasive but wrong reasoning. Cost and latency rise with long prompts and extended reasoning. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Hardware fit does not specify production throughput. Compare hosted, dedicated, and self-managed economics. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for high-value synthesis, multi-document comparison, query decomposition, and difficult answer generation after retrieval. Long context is not a replacement for retrieval, access control, citations, or freshness checks. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[COHERE-MODELS] Cohere model catalog

<https://docs.cohere.com/docs/models> • accessed 2026-07-10

[COHERE-APLUS] Cohere Command A+

<https://docs.cohere.com/docs/command-a-plus> • accessed 2026-07-10

Command A

Model ID / alias: **command-a-03-2025** or current stable ID • Status: **Active enterprise model** • Archetype: **balanced**

Tags: balanced, rag, enterprise

Access	Cohere API and enterprise deployment
Modalities	Text
Advertised context	256K tokens
Advertised max output	8K tokens
Published pricing snapshot	\$2.50 input / \$10 output per 1M tokens

What it is and why it matters

Cohere positions Command A for tool use, agents, RAG, and multilingual work. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- tool use
- agents
- RAG
- multilingual generation
- enterprise workflows

Best-fit use cases

- enterprise RAG
- tool-using assistants
- multilingual search

When not to choose it first

- long-form output beyond 8K
- vision-only requirements

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. The 8K output limit is adequate for many grounded answers but not full long reports; split or stream output. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[COHERE-MODELS] Cohere model catalog
<https://docs.cohere.com/docs/models> • accessed 2026-07-10

[COHERE-A] Cohere Command A
<https://docs.cohere.com/docs/command-a> • accessed 2026-07-10

Command A Reasoning

Model ID / alias: **Current Command A Reasoning ID** • Status: **Active reasoning model** • Archetype: **reasoning**

Tags: reasoning, enterprise, rag

Access	Cohere API and enterprise options
Modalities	Text
Advertised context	256K tokens
Advertised max output	32K tokens
Published pricing snapshot	Verify live pricing

What it is and why it matters

Cohere lists a dedicated Command A Reasoning model with 256K context and 32K output. Optimized for deliberate multi-step reasoning, planning, coding, mathematics, and hard analytical tasks where extra inference work can improve reliability.

Core capabilities

- deliberate reasoning
- multi-step agents
- analytical generation
- tool use

Best-fit use cases

- complex enterprise analysis
- hard RAG synthesis
- planning

When not to choose it first

- simple traffic where reasoning overhead is unnecessary

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated reasoning capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Reasoning tokens increase cost and latency and can still produce a confident wrong path. Internal reasoning is not auditable evidence. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Measure whether reasoning improves correctness on your corpus, not only verbosity or apparent sophistication. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for evidence reconciliation, retrieval planning, and analytical answers. Require source-linked conclusions and use lower-cost models for easy requests. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[COHERE-MODELS] Cohere model catalog

<https://docs.cohere.com/docs/models> • accessed 2026-07-10

Embed v4.0

Model ID / alias: embed-v4.0 • Status: Active embedding model • Archetype: embedding

Tags: embedding, multimodal, rag

Access	Cohere API and enterprise deployment
Modalities	Text, images, and mixed-content documents
Advertised context	128K tokens
Advertised max output	256/512/1024/1536-dimensional vectors
Published pricing snapshot	Verify current Cohere pricing

What it is and why it matters

Cohere lists Embed v4.0 for text, images, and mixed PDFs with selectable vector dimensions. Maps content into vectors for semantic retrieval, clustering, recommendations, deduplication, and similarity search.

Core capabilities

- multimodal embeddings
- PDF and document search
- semantic retrieval
- selectable dimensions

Best-fit use cases

- multimodal RAG
- enterprise search
- image-text retrieval

When not to choose it first

- index migration without re-embedding

- vector-only search for exact terms

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated embedding capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Embedding similarity is not truth or entailment. Scores vary by language, domain, chunk length, query style, and index configuration. Vector-only search misses exact identifiers. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Dimension is a storage, latency, and quality tradeoff. Evaluate the lowest dimension that meets recall targets. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Combine dense retrieval with lexical search, metadata filters, and reranking. Evaluate recall@k and record model version and dimensions in index metadata. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[COHERE-MODELS] Cohere model catalog

<https://docs.cohere.com/docs/models> • accessed 2026-07-10

Rerank v4 Pro

Model ID / alias: Current rerank-v4-pro ID • Status: Active reranker • Archetype: reranker

Tags: reranker, rag, search

Access	Cohere API
Modalities	Query and document pairs
Advertised context	32K tokens
Advertised max output	Relevance scores/ranking
Published pricing snapshot	Request or token dependent; verify live pricing

What it is and why it matters

Cohere lists Rerank v4 Pro as a high-quality reranker with a 32K context allowance. Rerankers score query-document pairs more precisely than first-stage vector similarity and improve evidence selection.

Core capabilities

- high-accuracy reranking
- enterprise search
- evidence selection
- multilingual ranking

Best-fit use cases

- second-stage RAG retrieval
- search improvement
- citation selection

When not to choose it first

- reranking thousands of full documents
- using score as factual confidence

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated reranker capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

They add latency and cost and can still prefer a fluent but incomplete passage. Context limits constrain candidate length and count. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Use first-stage retrieval to form a candidate set; reranking is not a substitute for indexing and filters. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Retrieve broadly, rerank a manageable candidate set, diversify evidence, and measure answer accuracy rather than only reranker score. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[COHERE-MODELS] Cohere model catalog

<https://docs.cohere.com/docs/models> • accessed 2026-07-10

8. xAI

Grok APIs target large-context reasoning, tool use, current-information workflows, and voice. The portfolio includes flagship and more economical variants. Evaluate safety, enterprise controls, regional access, and whether bundled or external search meets source-quality requirements.

Models covered

- Grok 4.5 (grok-4.5) — Active flagship; frontier; Text and supported multimodal/tool interfaces.
- Grok 4.3 (grok-4.3) — Active lower-cost model; balanced; Text and supported tools.
- xAI Voice Agent (Current voice-agent model) — Active specialized service; audio; Speech and text.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Grok 4.5

Model ID / alias: grok-4.5 • **Status:** Active flagship • **Archetype:** frontier

Tags: frontier, agent, coding

Access	xAI API
Modalities	Text and supported multimodal/tool interfaces
Advertised context	500K tokens
Advertised max output	See current catalog
Published pricing snapshot	\$2 input / \$6 output per 1M tokens

What it is and why it matters

xAI lists Grok 4.5 as its flagship and describes configurable reasoning and strong agentic/coding capability. Strong general reasoning, instruction following, coding, synthesis, and tool orchestration make it suitable as a primary generator or agent brain.

Core capabilities

- reasoning
- coding
- agentic tools
- configurable reasoning
- general generation

Best-fit use cases

- cost-competitive agents
- coding
- long-context analysis

When not to choose it first

- assuming vendor claims remove verification
- current-event questions without search tools

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated frontier capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Frontier capability does not eliminate hallucination. It can invent citations, make invalid assumptions, over-trust retrieved text, or produce persuasive but wrong reasoning. Cost and latency rise with long prompts and extended reasoning. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. xAI documentation notes model knowledge is not a live event feed. Use search for current information and verify citations. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for high-value synthesis, multi-document comparison, query decomposition, and difficult answer generation after retrieval. Long context is not a replacement for retrieval, access control, citations, or freshness checks. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[XAI-MODELS] xAI model catalog

<https://docs.x.ai/developers/models> • accessed 2026-07-10

[XAI-PRICING] xAI API pricing

<https://docs.x.ai/developers/pricing> • accessed 2026-07-10

Grok 4.3

Model ID / alias: **grok-4.3** • Status: **Active lower-cost model** • Archetype: **balanced**

Tags: balanced, long-context, agent

Access	xAI API
Modalities	Text and supported tools
Advertised context	1M tokens
Advertised max output	See current catalog
Published pricing snapshot	\$1.25 input / \$2.50 output per 1M tokens

What it is and why it matters

xAI pricing lists Grok 4.3 with a 1M context window and lower rates than Grok 4.5. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- long-context generation
- reasoning
- tool use
- general assistants

Best-fit use cases

- cost-aware long-context RAG
- agents
- large-document analysis

When not to choose it first

- unfiltered million-token prompts
- using context size as a quality benchmark

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. At very long context, retrieval selection and prompt organization still matter. Test evidence at different positions. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[XAI-MODELS] xAI model catalog
<https://docs.x.ai/developers/models> • accessed 2026-07-10

[XAI-PRICING] xAI API pricing
<https://docs.x.ai/developers/pricing> • accessed 2026-07-10

xAI Voice Agent

Model ID / alias: Current voice-agent model • **Status:** Active specialized service • **Archetype:** audio

Tags: audio, voice, agent

Access	xAI voice API
Modalities	Speech and text
Advertised context	Session dependent
Advertised max output	Streaming speech/text
Published pricing snapshot	\$3 per hour for listed voice agent; speech services priced separately

What it is and why it matters

xAI publishes separate voice-agent, text-to-speech, and speech-to-text prices. Supports speech input/output or realtime dialogue for voice agents, transcription, translation, and multimodal interfaces.

Core capabilities

- voice conversation
- speech synthesis
- speech recognition
- tool-enabled agents

Best-fit use cases

- voice automation
- interactive support
- spoken copilots

When not to choose it first

- cost estimates based only on text tokens
- high-risk actions without confirmation

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated audio capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Noise, accents, crosstalk, latency, turn-taking, numbers, names, and domain terminology remain frequent failure points. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Voice economics also include telephony, streaming infrastructure, silence, retries, and human transfer. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Transcribe and normalize the query, retrieve text evidence, then generate speech. Keep a transcript, expose citations visually, and confirm high-risk entities and numbers. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources**[XAI-MODELS] xAI model catalog**

<https://docs.x.ai/developers/models> • accessed 2026-07-10

[XAI-PRICING] xAI API pricing

<https://docs.x.ai/developers/pricing> • accessed 2026-07-10

9. DeepSeek

DeepSeek offers highly cost-competitive reasoning and general API models with very long context. Aggressive pricing is attractive for experimentation and routing. Endpoint aliases, deprecation dates, data policy, regional compliance, and behavioral changes should be monitored closely.

Models covered

- DeepSeek V4 Pro (deepseek-v4-pro) — Active; older chat/reasoner aliases scheduled for deprecation 2026-07-24; reasoning; Text/code.
- DeepSeek V4 Flash (deepseek-v4-flash) — Active; fast; Text/code.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

DeepSeek V4 Pro

Model ID / alias: deepseek-v4-pro • **Status:** Active; older chat/reasoner aliases scheduled for deprecation 2026-07-24 • **Archetype:** reasoning

Tags: reasoning, coding, long-context, cost

Access	DeepSeek API and released open artifacts
Modalities	Text/code
Advertised context	1M tokens
Advertised max output	384K tokens
Published pricing snapshot	\$0.003625 cache-hit input / \$0.435 cache-miss input / \$0.87 output per 1M tokens

What it is and why it matters

DeepSeek lists V4 Pro with 1M context, 384K maximum output, thinking and non-thinking modes, and tool/JSON support. Optimized for deliberate multi-step reasoning, planning, coding, mathematics, and hard analytical tasks where extra inference work can improve reliability.

Core capabilities

- thinking and non-thinking modes
- agentic coding
- tool calls
- JSON output
- prefix completion

Best-fit use cases

- cost-sensitive reasoning
- coding agents
- very long output workflows

When not to choose it first

- failing to migrate aliases
- assuming cache-hit price for every request
- unbounded output

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated reasoning capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Reasoning tokens increase cost and latency and can still produce a confident wrong path. Internal reasoning is not auditable evidence. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Separate cache-hit and cache-miss tokens in cost dashboards. Set hard output limits despite the large maximum. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for evidence reconciliation, retrieval planning, and analytical answers. Require source-linked conclusions and use lower-cost models for easy requests. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[DEEPSEEK-PRICE] DeepSeek API pricing

https://api-docs.deepseek.com/quick_start/pricing/ • accessed 2026-07-10

[DEEPSEEK-V4] DeepSeek V4 release

<https://api-docs.deepseek.com/news/news260424> • accessed 2026-07-10

DeepSeek V4 Flash

Model ID / alias: **deepseek-v4-flash** • Status: **Active** • Archetype: **fast**

Tags: fast, cost, coding

Access	DeepSeek API and released artifacts
Modalities	Text/code
Advertised context	1M tokens
Advertised max output	384K tokens
Published pricing snapshot	\$0.0028 cache-hit input / \$0.14 cache-miss input / \$0.28 output per 1M tokens

What it is and why it matters

DeepSeek lists V4 Flash as a lower-cost V4 endpoint with the same headline context and output limits. Designed for high throughput, low latency, and lower unit cost. It is useful for query rewriting, metadata extraction, routing, classification, and first-pass answers.

Core capabilities

- low-cost generation
- thinking and non-thinking modes
- tool calls
- JSON output
- coding

Best-fit use cases

- high-volume RAG
- code utilities
- routing
- batch processing

When not to choose it first

- assuming low price guarantees SLA or enterprise fit
- unlimited long outputs

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated fast capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Fast models are more sensitive to ambiguity and retrieval noise. They can miss contradictions, omit qualifiers, or follow a plausible distractor. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Evaluate availability, data policy, jurisdiction, support, and rate limits in addition to raw price. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for utility stages or simple grounded Q&A. Escalate difficult and high-risk questions rather than forcing one cheap model to handle every request. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources**[DEEPSEEK-PRICE] DeepSeek API pricing**

https://api-docs.deepseek.com/quick_start/pricing/ • accessed 2026-07-10

[DEEPSEEK-V4] DeepSeek V4 release

<https://api-docs.deepseek.com/news/news260424> • accessed 2026-07-10

10. Alibaba / Qwen

Qwen spans flagship, fast, coding, vision, audio, embedding, and reranking models, with open-weight and managed options. It is particularly relevant for multilingual and Chinese-language workloads. International and mainland-China endpoints, pricing, and availability can differ.

Models covered

- Qwen 3.7 Max (qwen3.7-max) — Active recommended flagship; frontier; Text and endpoint-supported multimodal/tool capabilities.
- Qwen 3.7 Plus (qwen3.7-plus) — Active balanced model; balanced; Model-page dependent.
- Qwen 3.6 Flash (qwen3.6-flash) — Active efficiency model; fast; Endpoint dependent.
- Qwen3-Coder (Qwen3-Coder family) — Open and hosted coding family; open; Text/code.
- Qwen3 Embedding (Qwen3-Embedding family) — Open/hosted embedding family; embedding; Text to vector.
- Qwen3 Reranker (Qwen3-Reranker family) — Open/hosted reranker family; reranker; Query-document pairs.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Qwen 3.7 Max

Model ID / alias: qwen3.7-max • **Status:** Active recommended flagship • **Archetype:** frontier

Tags: frontier, reasoning, agent, long-context

Access	Alibaba Cloud Model Studio; availability varies by region
Modalities	Text and endpoint-supported multimodal/tool capabilities
Advertised context	Up to 1M input tier
Advertised max output	See live catalog
Published pricing snapshot	\$2.50 input / \$7.50 output per 1M tokens at listed 1M tier; tiering and cache can change cost

What it is and why it matters

Alibaba Cloud recommends Qwen 3.7 Max as its current flagship model. Strong general reasoning, instruction following, coding, synthesis, and tool orchestration make it suitable as a primary generator or agent brain.

Core capabilities

- thinking and non-thinking modes
- advanced reasoning
- agents
- coding
- long context

Best-fit use cases

- Alibaba Cloud enterprise agents
- multilingual RAG
- complex analysis

When not to choose it first

- using one tier price for every prompt
- cross-region assumptions

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated frontier capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Frontier capability does not eliminate hallucination. It can invent citations, make invalid assumptions, over-trust retrieved text, or produce persuasive but wrong reasoning. Cost and latency rise with long prompts and extended reasoning. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Pricing is tiered for parts of the family. Capture

prompt tier, region, and effective date in usage logs. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for high-value synthesis, multi-document comparison, query decomposition, and difficult answer generation after retrieval. Long context is not a replacement for retrieval, access control, citations, or freshness checks. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[QWEN-MODELS] Alibaba Cloud Model Studio model catalog

<https://www.alibabacloud.com/help/en/model-studio/models> • accessed 2026-07-10

[QWEN-PRICE] Alibaba Cloud Model Studio pricing

<https://www.alibabacloud.com/help/en/model-studio/model-pricing> • accessed 2026-07-10

Qwen 3.7 Plus

Model ID / alias: **qwen3.7-plus** • Status: **Active balanced model** • Archetype: **balanced**

Tags: balanced, multilingual, agent

Access	Alibaba Cloud Model Studio
Modalities	Model-page dependent
Advertised context	Large context; verify catalog
Advertised max output	See live catalog
Published pricing snapshot	Tiered; verify current Model Studio pricing

What it is and why it matters

Alibaba lists Qwen 3.7 Plus among its recommended current models. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- general assistants
- reasoning
- tool use
- multilingual generation
- enterprise workloads

Best-fit use cases

- default Qwen enterprise assistant
- RAG generation
- agents

When not to choose it first

- hard-coding prices or regional availability

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Evaluate English and each target language separately; multilingual averages hide language-specific weaknesses. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[QWEN-MODELS] Alibaba Cloud Model Studio model catalog

<https://www.alibabacloud.com/help/en/model-studio/models> • accessed 2026-07-10

[QWEN-PRICE] Alibaba Cloud Model Studio pricing

<https://www.alibabacloud.com/help/en/model-studio/model-pricing> • accessed 2026-07-10

Qwen 3.6 Flash

Model ID / alias: **qwen3.6-flash** • Status: **Active efficiency model** • Archetype: **fast**

Tags: fast, multilingual

Access	Alibaba Cloud Model Studio
Modalities	Endpoint dependent
Advertised context	See live catalog
Advertised max output	See live catalog
Published pricing snapshot	Verify tiered pricing

What it is and why it matters

Alibaba includes Qwen 3.6 Flash in its recommended lineup. Designed for high throughput, low latency, and lower unit cost. It is useful for query rewriting, metadata extraction, routing, classification, and first-pass answers.

Core capabilities

- high-throughput generation
- classification
- translation
- agent utilities

Best-fit use cases

- pipeline utilities
- bulk extraction
- low-latency assistants

When not to choose it first

- complex reasoning without escalation
- long noisy contexts

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated fast capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Fast models are more sensitive to ambiguity and retrieval noise. They can miss contradictions, omit qualifiers, or follow a plausible distractor. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Use task-based confidence and validation rather than trusting self-reported confidence. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for utility stages or simple grounded Q&A. Escalate difficult and high-risk questions rather than forcing one cheap model to handle every request. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[QWEN-MODELS] Alibaba Cloud Model Studio model catalog

<https://www.alibabacloud.com/help/en/model-studio/models> • accessed 2026-07-10

[QWEN-PRICE] Alibaba Cloud Model Studio pricing

<https://www.alibabacloud.com/help/en/model-studio/model-pricing> • accessed 2026-07-10

Qwen3-Coder

Model ID / alias: Qwen3-Coder family • **Status:** Open and hosted coding family • **Archetype:** open

Tags: open, coding, agent

Access	Open weights and Alibaba/partner endpoints
Modalities	Text/code
Advertised context	256K native; announcement discusses extrapolation to 1M for 480B-A35B
Advertised max output	Deployment dependent
Published pricing snapshot	Self-hosting or endpoint dependent

What it is and why it matters

Qwen introduced a 480B-total, 35B-active coding model with 256K native context and longer extrapolated context. Open-weight access provides control over hosting, quantization, fine-tuning, data locality, observability, and economics at sustained volume.

Core capabilities

- agentic coding
- repository-level work
- tool use
- long code context
- customization

Best-fit use cases

- coding agents
- private code RAG

- repository assistants

When not to choose it first

- treating extrapolated context as native quality
- merging code without tests

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated open capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

The deployer owns GPU capacity, serving optimization, upgrades, monitoring, abuse prevention, safety, and license compliance. Downloadable weights do not imply low total cost. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Measure issue resolution, patch minimality, test coverage, security, and license compliance on real repositories. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

A strong fit for private or regulated RAG when the organization can operate the stack. Benchmark the exact quantization, engine, prompt format, and hardware. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[QWEN-CODER] Qwen3-Coder announcement

<https://qwenlm.github.io/blog/qwen3-coder/> • accessed 2026-07-10

[QWEN-MODELS] Alibaba Cloud Model Studio model catalog

<https://www.alibabacloud.com/help/en/model-studio/models> • accessed 2026-07-10

Qwen3 Embedding

Model ID / alias: Qwen3-Embedding family • **Status:** Open/hosted embedding family • **Archetype:** embedding

Tags: embedding, open, multilingual

Access	Open models and Model Studio options
Modalities	Text to vector
Advertised context	Variant dependent
Advertised max output	Variant and dimension dependent
Published pricing snapshot	Self-hosting or hosted pricing

What it is and why it matters

Qwen released a dedicated Qwen3 embedding and reranking family. Maps content into vectors for semantic retrieval, clustering, recommendations, deduplication, and similarity search.

Core capabilities

- semantic search
- multilingual retrieval
- code retrieval

- RAG indexing

Best-fit use cases

- multilingual RAG
- code search
- private vector indexes

When not to choose it first

- deployment without language-specific recall evaluation

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated embedding capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Embedding similarity is not truth or entailment. Scores vary by language, domain, chunk length, query style, and index configuration. Vector-only search misses exact identifiers. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. A multilingual index needs balanced evaluation in every important language, including code-switching and transliteration. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Combine dense retrieval with lexical search, metadata filters, and reranking. Evaluate recall@k and record model version and dimensions in index metadata. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[QWEN-EMBED] Qwen3 Embedding and Reranker
<https://qwenlm.github.io/blog/qwen3-embedding/> • accessed 2026-07-10

[QWEN-MODELS] Alibaba Cloud Model Studio model catalog
<https://www.alibabacloud.com/help/en/model-studio/models> • accessed 2026-07-10

Qwen3 Reranker

Model ID / alias: Qwen3-Reranker family • Status: Open/hosted reranker family • Archetype: reranker
Tags: reranker, open, multilingual

Access	Open artifacts and hosted options
Modalities	Query-document pairs
Advertised context	Variant dependent
Advertised max output	Relevance scores
Published pricing snapshot	Deployment dependent

What it is and why it matters

Qwen released rerankers alongside Qwen3 embeddings. Rerankers score query-document pairs more precisely than first-stage vector similarity and improve evidence selection.

Core capabilities

- multilingual reranking
- code/text relevance
- RAG evidence selection

Best-fit use cases

- hybrid retrieval
- multilingual enterprise search

When not to choose it first

- feeding whole repositories as candidates
- ignoring latency at high candidate counts

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated reranker capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

They add latency and cost and can still prefer a fluent but incomplete passage. Context limits constrain candidate length and count. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Tune candidate count, rerank batch size, and document truncation together because each affects recall and latency. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Retrieve broadly, rerank a manageable candidate set, diversify evidence, and measure answer accuracy rather than only reranker score. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[QWEN-EMBED] **Qwen3 Embedding and Reranker**

<https://qwenlm.github.io/blog/qwen3-embedding/> • accessed 2026-07-10

[QWEN-MODELS] **Alibaba Cloud Model Studio model catalog**

<https://www.alibabacloud.com/help/en/model-studio/models> • accessed 2026-07-10

11. Microsoft

Microsoft Foundry provides first-party Phi small models, model routing, and a catalog of partner models. Azure deployment choices include serverless and managed compute with region-specific availability. The platform is a governance and deployment layer as much as a model family.

Models covered

- Phi-4 (Phi-4) — Active compact model in Foundry catalog; small; Text.
- Phi-4-mini-instruct (Phi-4-mini-instruct) — Active small instruction model; small; Text.
- Phi-4-reasoning (Phi-4-reasoning) — Active reasoning model in supported regions; reasoning; Text.
- Phi-4-multimodal-instruct (Phi-4-multimodal-instruct) — Active compact multimodal model; vision; Text and vision plus model-supported inputs.
- Microsoft Model Router (Current Foundry Model Router) — Active platform capability/model group; balanced; Depends on routed models.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Phi-4

Model ID / alias: Phi-4 • **Status:** Active compact model in Foundry catalog • **Archetype:** small

Tags: small, microsoft, enterprise

Access	Microsoft Foundry serverless/managed compute and ecosystem runtimes
Modalities	Text
Advertised context	Model-card dependent
Advertised max output	Deployment dependent
Published pricing snapshot	Deployment and region dependent

What it is and why it matters

Microsoft Foundry lists Phi-4 among Microsoft models available in its catalog. Small language models can run cheaply, privately, and sometimes at the edge. They are effective for narrow extraction, routing, local assistants, and constrained domain tasks.

Core capabilities

- compact reasoning
- local or managed inference
- fine-tuning
- enterprise integration

Best-fit use cases

- narrow private assistants
- cost-controlled reasoning
- edge or server workloads

When not to choose it first

- open-ended frontier replacement
- assuming one region represents global availability

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated small capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

They have less world knowledge and reasoning headroom and are more likely to fail on compound instructions, rare concepts, multilingual edge cases, and conflicting evidence. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Foundry catalog and region tables change. Validate exact deployment type, region, quota, and fine-tuning support. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Keep the task narrow, retrieve concise evidence, use schemas, and validate outputs. A small model plus excellent retrieval can excel on a narrow domain but not every open-ended query. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[MS-DIRECT] Microsoft Foundry models sold directly by Azure

<https://learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/models-sold-directly-by-azure> • accessed 2026-07-10

[MS-REGIONS] Microsoft Foundry model region availability

<https://learn.microsoft.com/en-us/azure/foundry-classic/how-to/deploy-models-serverless-availability> • accessed 2026-07-10

Phi-4-mini-instruct

Model ID / alias: **Phi-4-mini-instruct** • Status: **Active small instruction model** • Archetype: **small**

Tags: small, edge, microsoft

Access	Microsoft Foundry and local/partner runtimes
Modalities	Text
Advertised context	Model-card dependent
Advertised max output	Deployment dependent
Published pricing snapshot	Deployment dependent

What it is and why it matters

Microsoft documentation uses Phi-4-mini-instruct in current fine-tuning and local deployment examples. Small language models can run cheaply, privately, and sometimes at the edge. They are effective for narrow extraction, routing, local assistants, and constrained domain tasks.

Core capabilities

- instruction following
- lightweight generation
- fine-tuning
- local CPU/GPU options

Best-fit use cases

- classification
- narrow RAG
- local assistants
- fine-tuned utilities

When not to choose it first

- complex long-horizon agents
- very large evidence sets

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated small capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

They have less world knowledge and reasoning headroom and are more likely to fail on compound instructions, rare concepts, multilingual edge cases, and conflicting evidence. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Test the exact runtime artifact; CPU, ONNX, quantized, serverless, and managed variants can behave differently. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Keep the task narrow, retrieve concise evidence, use schemas, and validate outputs. A small model plus excellent retrieval can excel on a narrow domain but not every open-ended query. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[MS-DIRECT] Microsoft Foundry models sold directly by Azure

<https://learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/models-sold-directly-by-azure> • accessed 2026-07-10

[MS-REGIONS] Microsoft Foundry model region availability

<https://learn.microsoft.com/en-us/azure/foundry-classic/how-to/deploy-models-serverless-availability> • accessed 2026-07-10

Phi-4-reasoning

Model ID / alias: Phi-4-reasoning • **Status:** Active reasoning model in supported regions • **Archetype:** reasoning

Tags: small, reasoning, microsoft

Access	Microsoft Foundry; availability varies
Modalities	Text
Advertised context	See model card
Advertised max output	See endpoint
Published pricing snapshot	Deployment dependent

What it is and why it matters

Microsoft availability documentation lists Phi-4-reasoning and Phi-4-mini-reasoning in specified Foundry regions. Optimized for deliberate multi-step reasoning, planning, coding, mathematics, and hard analytical tasks where extra inference work can improve reliability.

Core capabilities

- compact reasoning
- mathematics and analytical tasks
- managed enterprise deployment

Best-fit use cases

- bounded analytical tasks
- cost-aware reasoning
- private enterprise workflows

When not to choose it first

- assuming compact reasoning equals frontier breadth
- unsupported regions

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated reasoning capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Reasoning tokens increase cost and latency and can still produce a confident wrong path. Internal reasoning is not auditable evidence. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Reasoning SLMs are strong routing candidates, but hard domain questions still need evidence-linked evaluation. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for evidence reconciliation, retrieval planning, and analytical answers. Require source-linked conclusions and use lower-cost models for easy requests. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[MS-REGIONS] Microsoft Foundry model region availability
<https://learn.microsoft.com/en-us/azure/foundry-classic/how-to/deploy-models-serverless-availability> • accessed 2026-07-10

[MS-DIRECT] Microsoft Foundry models sold directly by Azure
<https://learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/models-sold-directly-by-azure> • accessed 2026-07-10

Phi-4-multimodal-instruct

Model ID / alias: **Phi-4-multimodal-instruct** • Status: **Active compact multimodal model** • Archetype: **vision**

Tags: small, vision, microsoft

Access	Microsoft Foundry and supported runtimes
Modalities	Text and vision plus model-supported inputs
Advertised context	See model card
Advertised max output	See endpoint
Published pricing snapshot	Deployment dependent

What it is and why it matters

Microsoft lists Phi-4-multimodal-instruct in current Foundry availability documentation. Accepts images or document pages and can reason over charts, screenshots, diagrams, and mixed text-image inputs.

Core capabilities

- compact multimodal understanding
- document and image assistance
- enterprise deployment

Best-fit use cases

- visual document assistants
- local multimodal prototypes
- cost-controlled image Q&A

When not to choose it first

- dense OCR without a parser
- specialized high-resolution vision without testing

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated vision capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Vision models can misread small text, dense tables, handwriting, rotated pages, fine spatial relationships, and low-resolution scans. They may describe plausible objects that are absent. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Compare a parser-plus-text-model pipeline with direct page-image inference for document RAG. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

For document RAG, use layout/OCR extraction and preserve page images. Use visual reasoning where the answer depends on layout or imagery, not as the only ingestion method. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[MS-REGIONS] Microsoft Foundry model region availability
<https://learn.microsoft.com/en-us/azure/foundry-classic/how-to/deploy-models-serverless-availability> • accessed 2026-07-10

[MS-DIRECT] Microsoft Foundry models sold directly by Azure
<https://learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/models-sold-directly-by-azure> • accessed 2026-07-10

Microsoft Model Router

Model ID / alias: Current Foundry Model Router • Status: Active platform capability/model group • Archetype: balanced

Tags: router, platform, enterprise

Access	Microsoft Foundry
Modalities	Depends on routed models
Advertised context	Depends on route
Advertised max output	Depends on route
Published pricing snapshot	Routing and underlying billing depend on offering

What it is and why it matters

Microsoft lists Model Router among its Foundry model groups. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- automatic model selection
- cost-quality routing
- unified enterprise access
- operational simplification

Best-fit use cases

- mixed-complexity workloads
- enterprise portfolios
- gradual migration

When not to choose it first

- opaque routing without logs and evaluation
- fixed-model regulated tasks unless constrained

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. A router is a policy system. Define allowed models, data regions, latency budgets, fallbacks, and per-route evaluation. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[MS-DIRECT] Microsoft Foundry models sold directly by Azure

<https://learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/models-sold-directly-by-azure> • accessed 2026-07-10

12. NVIDIA

Nemotron models and NIM/NeMo tooling target open, accelerated enterprise AI across reasoning, agents, and vision. NVIDIA's stack is compelling for organizations operating GPU infrastructure. Total cost depends on utilization, quantization, inference engine, and operational maturity.

Models covered

- Nemotron 3 Ultra 550B-A55B (nvidia-nemotron-3-ultra) — Current open-weight reasoning model; frontier; Text.
- Nemotron 3 Super (nvidia-nemotron-3-super) — Current open-weight enterprise model; balanced; Text.
- Nemotron 3 Nano (nvidia-nemotron-3-nano) — Current compact open-weight model; small; Text.
- Nemotron Nano VL (nemotron-nano-vl) — Current compact vision-language model; vision; Text + images / visual documents.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Nemotron 3 Ultra 550B-A55B

Model ID / alias: nvidia-nemotron-3-ultra • **Status:** Current open-weight reasoning model • **Archetype:** frontier

Tags: open-weight, reasoning, agents, enterprise, self-hosted

Access	NVIDIA NIM / model repositories / partner clouds; verify regional availability
Modalities	Text
Advertised context	Provider/model-card dependent; verify the deployed release
Advertised max output	Provider/model-card dependent
Published pricing snapshot	Self-hosting infrastructure cost or partner-platform metering; no single universal price

What it is and why it matters

NVIDIA positions Nemotron 3 Ultra as the largest reasoning member of the Nemotron 3 family. Strong general reasoning, instruction following, coding, synthesis, and tool orchestration make it suitable as a primary generator or agent brain.

Core capabilities

- Large mixture-of-experts model aimed at advanced reasoning, agentic workflows, coding, mathematics, and enterprise AI.
- Designed for deployment through NVIDIA's accelerated inference ecosystem and optimized serving stack.
- Open-weight access gives organizations more control over placement, tuning, and observability than a closed API.

Best-fit use cases

- High-complexity reasoning and agent systems where self-hosting or NVIDIA-aligned deployment is strategic.
- Organizations already standardized on NVIDIA GPUs, NIM microservices, NeMo tooling, or sovereign infrastructure.
- Research and controlled enterprise deployments that need access to model weights.

When not to choose it first

- Small-budget projects that cannot operate high-end GPU infrastructure.
- Latency-sensitive applications where a much smaller model can meet the quality bar.
- Teams seeking a fully managed, provider-agnostic API with minimal operations.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated frontier capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Frontier capability does not eliminate hallucination. It can invent citations, make invalid assumptions, over-trust retrieved text, or produce persuasive but wrong reasoning. Cost and latency rise with long prompts and extended reasoning. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. The headline parameter count does not translate directly to per-token compute in a mixture-of-experts architecture. Measure actual throughput, memory, quality, and total

operating cost on the intended stack. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for high-value synthesis, multi-document comparison, query decomposition, and difficult answer generation after retrieval. Long context is not a replacement for retrieval, access control, citations, or freshness checks. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[NVIDIA-ULTRA] **NVIDIA Nemotron 3 Ultra model card**
<https://build.nvidia.com/nvidia/nemotron-3-ultra-550b-a55b/modelcard> • accessed 2026-07-10

[NVIDIA-SUPER] **NVIDIA Nemotron 3 Super**
<https://build.nvidia.com/nvidia/nemotron-3-super-120b-a12b> • accessed 2026-07-10

Nemotron 3 Super

Model ID / alias: nvidia-nemotron-3-super • **Status:** Current open-weight enterprise model • **Archetype:** balanced

Tags: open-weight, long-context, agents, enterprise, rag

Access	NVIDIA ecosystem and selected partner platforms
Modalities	Text
Advertised context	Up to 1M tokens in selected deployments; verify the model card and endpoint
Advertised max output	Endpoint-dependent
Published pricing snapshot	Example managed-platform rates vary; self-hosted cost depends on GPU utilization

What it is and why it matters

NVIDIA documentation highlights a one-million-token context window for Nemotron 3 Super in supported deployments. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- Enterprise-oriented reasoning and agentic model positioned between Ultra and Nano.
- Long-context operation for document-heavy agents and repository-scale workflows.
- Designed to balance model quality with more practical deployment economics than the largest family member.

Best-fit use cases

- Long-context enterprise RAG, document analysis, coding assistants, and tool-using agents.
- Private deployments that need stronger quality than a compact model but cannot justify the largest model.
- NVIDIA-accelerated inference environments.

When not to choose it first

- Extremely low-latency edge workloads.
- Use cases where managed frontier APIs are operationally simpler and data policy permits them.
- Projects that have not benchmarked long-context retrieval quality and GPU cost.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Long context is not a substitute for retrieval. Test evidence recall at different positions and compare against retrieval-plus-reranking. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[NVIDIA-ULTRA] NVIDIA Nemotron 3 Ultra model card

<https://build.nvidia.com/nvidia/nemotron-3-ultra-550b-a55b/modelcard> • accessed 2026-07-10

[NVIDIA-SUPER] NVIDIA Nemotron 3 Super

<https://build.nvidia.com/nvidia/nemotron-3-super-120b-a12b> • accessed 2026-07-10

Nemotron 3 Nano

Model ID / alias: nvidia-nemotron-3-nano • **Status:** Current compact open-weight model • **Archetype:** small

Tags: open-weight, small-model, routing, extraction, cost

Access	NVIDIA ecosystem / partner platforms / self-hosting
Modalities	Text
Advertised context	Release-dependent; verify model card
Advertised max output	Release-dependent
Published pricing snapshot	Low managed-platform token rates are available on some clouds; self-hosted TCO depends on hardware

What it is and why it matters

NVIDIA positions Nano as the efficient member of the Nemotron 3 family. Small language models can run cheaply, privately, and sometimes at the edge. They are effective for narrow extraction, routing, local assistants, and constrained domain tasks.

Core capabilities

- Compact reasoning and instruction-following model intended for efficient agent components and high-volume inference.
- Can be used as a router, extractor, classifier, summarizer, or first-pass answer model.
- Open-weight deployment supports private and cost-controlled architectures.

Best-fit use cases

- High-volume classification, extraction, routing, lightweight RAG, and edge-adjacent server inference.
- Cascaded systems that escalate difficult prompts to a larger model.
- Cost-sensitive private deployments.

When not to choose it first

- Tasks requiring frontier-level world knowledge or difficult multi-step reasoning without escalation.
- High-stakes answers that are not grounded and independently checked.
- Very long, highly nuanced document synthesis without careful evaluation.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated small capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

They have less world knowledge and reasoning headroom and are more likely to fail on compound instructions, rare concepts, multilingual edge cases, and conflicting evidence. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Small models often perform exceptionally well on narrow, structured tasks after prompt tuning or fine-tuning, but may fail abruptly outside the tested distribution. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Keep the task narrow, retrieve concise evidence, use schemas, and validate outputs. A small model plus excellent retrieval can excel on a narrow domain but not every open-ended query. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[NVIDIA-ULTRA] NVIDIA Nemotron 3 Ultra model card

<https://build.nvidia.com/nvidia/nemotron-3-ultra-550b-a55b/modelcard> • accessed 2026-07-10

[NVIDIA-SUPER] NVIDIA Nemotron 3 Super

<https://build.nvidia.com/nvidia/nemotron-3-super-120b-a12b> • accessed 2026-07-10

Nemotron Nano VL

Model ID / alias: nemotron-nano-vl • **Status:** Current compact vision-language model • **Archetype:** vision

Tags: vision, open-weight, documents, multimodal, private

Access	NVIDIA model ecosystem and compatible self-hosted serving
Modalities	Text + images / visual documents
Advertised context	Release-dependent
Advertised max output	Release-dependent
Published pricing snapshot	Self-hosted or partner-platform pricing

What it is and why it matters

NVIDIA presents Nano VL as a compact visual-language member of its enterprise model ecosystem. Accepts images or document pages and can reason over charts, screenshots, diagrams, and mixed text-image inputs.

Core capabilities

- Compact vision-language understanding for images, screenshots, charts, and document pages.
- Useful as a visual extraction or triage component before text generation.

- Can support private document processing where source images cannot leave controlled infrastructure.

Best-fit use cases

- Visual document classification, screenshot understanding, chart triage, and multimodal RAG preprocessing.
- Private or on-premises visual pipelines.
- Cost-aware multimodal agents.

When not to choose it first

- Pixel-perfect OCR or layout reconstruction without a dedicated OCR/document parser.
- Medical, legal, or industrial visual decisions without specialized validation.
- Tasks where tiny graphical details determine the answer and resolution handling is uncertain.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated vision capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Vision models can misread small text, dense tables, handwriting, rotated pages, fine spatial relationships, and low-resolution scans. They may describe plausible objects that are absent. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Vision-language models can confidently misread small text, axes, legends, or spatial relationships. Preserve the original page image and require evidence coordinates where possible. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

For document RAG, use layout/OCR extraction and preserve page images. Use visual reasoning where the answer depends on layout or imagery, not as the only ingestion method. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[NVIDIA-ULTRA] NVIDIA Nemotron 3 Ultra model card

<https://build.nvidia.com/nvidia/nemotron-3-ultra-550b-a55b/modelcard> • accessed 2026-07-10

13. IBM

Granite models, embeddings, vision, and Docling emphasize enterprise governance, hybrid deployment, and document-centric AI. Open components support controlled environments. The exact Granite variant and document-processing pipeline should be pinned for reproducibility.

Models covered

- Granite 4.1 (granite-4.1) — Current Granite generation; balanced; Text.
- Granite 4.0 (granite-4.0) — Previous/currently available Granite generation; balanced; Text.
- Granite Vision (granite-vision) — Current visual-document model family; vision; Text + images / document pages.
- Granite Embedding (granite-embedding) — Current embedding family; embedding; Text.
- Docling (docling) — Current open document-processing toolkit/model ecosystem; ocr; PDFs, images, office-style documents and structured text outputs.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Granite 4.1

Model ID / alias: granite-4.1 • **Status:** Current Granite generation • **Archetype:** balanced

Tags: open-weight, enterprise, governance, rag, hybrid-cloud

Access	IBM watsonx / open model repositories / supported partner platforms
Modalities	Text
Advertised context	Variant-dependent
Advertised max output	Variant-dependent
Published pricing snapshot	Platform or infrastructure dependent

What it is and why it matters

IBM positions Granite as an enterprise-focused, open model family. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- Enterprise-oriented instruction following, RAG, summarization, extraction, coding, and agent workflows.
- Open model variants support private deployment and transparent evaluation.
- Granite releases emphasize business use, governance, efficiency, and integration with IBM's enterprise AI stack.

Best-fit use cases

- Governed enterprise RAG and workflow automation.
- Organizations using watsonx, Red Hat, or hybrid-cloud deployment patterns.
- Private tuning and controlled deployment where open weights are preferred.

When not to choose it first

- Choosing solely on broad benchmark scores without testing domain terminology and document formats.
- Consumer chat applications that depend on a specific closed-provider ecosystem.
- Use cases requiring a modality not supported by the selected Granite variant.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Granite is a family, not one interchangeable endpoint. Record the exact variant, revision, quantization, license, context setting, and serving engine. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[IBM-GRANITE] IBM Granite family

<https://www.ibm.com/granite/docs/models/granite> • accessed 2026-07-10

Granite 4.0

Model ID / alias: granite-4.0 • **Status:** Previous/currently available Granite generation • **Archetype:** balanced

Tags: open-weight, enterprise, migration, rag, baseline

Access	IBM and open model distribution channels
Modalities	Text
Advertised context	Variant-dependent
Advertised max output	Variant-dependent
Published pricing snapshot	Platform or infrastructure dependent

What it is and why it matters

IBM's model catalog retains multiple Granite generations for different deployment and migration needs. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- General enterprise language tasks including retrieval-grounded generation, extraction, classification, and code-related work.
- Multiple sizes allow quality, latency, and infrastructure trade-offs.
- Suitable as a controlled baseline for migration testing against Granite 4.1.

Best-fit use cases

- Existing deployments that need stability while a newer release is evaluated.
- Private enterprise assistants and structured document processing.
- Regression baselines in a model-upgrade program.

When not to choose it first

- Starting a new deployment without comparing the newer supported generation.
- Assuming behavior is identical across sizes and instruction-tuned variants.
- Ignoring lifecycle and security updates.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance

across languages and domains. Pin revisions in production. A model-family name alone is insufficient for reproducibility. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[IBM-GRANITE] IBM Granite family

<https://www.ibm.com/granite/docs/models/granite> • accessed 2026-07-10

Granite Vision

Model ID / alias: granite-vision • **Status:** Current visual-document model family • **Archetype:** vision

Tags: vision, documents, open-weight, multimodal-rag, enterprise

Access	IBM ecosystem / open distribution where available
Modalities	Text + images / document pages
Advertised context	Variant-dependent
Advertised max output	Variant-dependent
Published pricing snapshot	Platform or infrastructure dependent

What it is and why it matters

IBM includes Granite Vision as a document-focused multimodal member of the Granite portfolio. Accepts images or document pages and can reason over charts, screenshots, diagrams, and mixed text-image inputs.

Core capabilities

- Visual document understanding for scanned pages, forms, tables, charts, and mixed-layout business documents.
- Can complement OCR and document parsing in multimodal RAG.
- Open deployment options can support controlled processing of sensitive documents.

Best-fit use cases

- Invoices, forms, reports, charts, and document-page question answering.
- Multimodal enterprise RAG where layout is semantically important.
- Private visual document pipelines.

When not to choose it first

- Replacing deterministic OCR for exact character-level transcription without validation.
- Using low-resolution page images when small text is critical.
- Assuming visual understanding automatically preserves table structure.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated vision capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Vision models can misread small text, dense tables, handwriting, rotated pages, fine spatial relationships, and low-resolution scans. They may describe plausible objects that are absent. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. For production document AI, combine visual reasoning with OCR confidence, layout coordinates, and source-page citations. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

For document RAG, use layout/OCR extraction and preserve page images. Use visual reasoning where the answer depends on layout or imagery, not as the only ingestion method. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[IBM-GRANITE] IBM Granite family

<https://www.ibm.com/granite/docs/models/granite> • accessed 2026-07-10

[IBM-DOCLING] IBM Granite Docling

<https://www.ibm.com/granite/docs/models/docling> • accessed 2026-07-10

Granite Embedding

Model ID / alias: granite-embedding • **Status:** Current embedding family • **Archetype:** embedding

Tags: embedding, retrieval, open-weight, rag, semantic-search

Access	IBM / open model repositories / supported platforms
Modalities	Text
Advertised context	Variant-dependent
Advertised max output	Dense vector representations
Published pricing snapshot	Platform or infrastructure dependent

What it is and why it matters

IBM provides Granite embedding models as part of its enterprise retrieval stack. Maps content into vectors for semantic retrieval, clustering, recommendations, deduplication, and similarity search.

Core capabilities

- Dense semantic embeddings for search, clustering, recommendations, deduplication, and RAG retrieval.
- Multiple sizes and language configurations may be available.
- Open deployment enables indexing inside a controlled environment.

Best-fit use cases

- Enterprise semantic search and private RAG indexes.
- Domain-specific retrieval evaluation and fine-tuning experiments.
- Hybrid retrieval combined with lexical search.

When not to choose it first

- Assuming an embedding model that performs well in English transfers equally to every language or domain.
- Changing embedding versions without re-embedding and measuring index compatibility.
- Using cosine similarity thresholds copied from an unrelated dataset.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated embedding capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Embedding similarity is not truth or entailment. Scores vary by language, domain, chunk length, query style, and index configuration. Vector-only search misses exact identifiers. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Evaluate Recall@k, MRR or nDCG on real queries. Keep the embedding model, preprocessing, pooling, normalization, and vector dimension in index metadata. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Combine dense retrieval with lexical search, metadata filters, and reranking. Evaluate recall@k and record model version and dimensions in index metadata. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[IBM-GRANITE] IBM Granite family

<https://www.ibm.com/granite/docs/models/granite> • accessed 2026-07-10

Docling

Model ID / alias: docling • **Status:** Current open document-processing toolkit/model ecosystem • **Archetype:** ocr

Tags: document-parsing, ocr, rag-ingestion, tables, open-source

Access	Open-source toolkit and IBM ecosystem
Modalities	PDFs, images, office-style documents and structured text outputs
Advertised context	Not a chat-model context window; processing is document/pipeline based
Advertised max output	Structured document representation, text, tables, layout and metadata
Published pricing snapshot	Open-source software; infrastructure and managed-platform costs apply

What it is and why it matters

IBM describes Docling as an open toolkit for document conversion and understanding. Converts visually rich documents into structured text or layout representations for indexing scans, tables, slides, and forms.

Core capabilities

- Document conversion and structural extraction for PDFs and other enterprise document formats.
- Preserves headings, tables, reading order, and page provenance more effectively than naive text extraction.
- Useful preprocessing layer for chunking, indexing, citations, and multimodal RAG.

Best-fit use cases

- RAG ingestion from complex PDFs, reports, manuals, and mixed-layout documents.
- Creating page-aware chunks with headings, table boundaries, and source metadata.
- Open, inspectable document pipelines.

When not to choose it first

- Treating every extracted table or reading order as perfect.

- Discarding original page images and coordinates before quality review.
- Using one chunking rule for narrative text, tables, code, and forms.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated ocr capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

OCR errors concentrate in small fonts, handwriting, merged cells, formulas, multi-column pages, and degraded scans. Structure recovery can be approximate. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Docling is not a general chat LLM, but it is often more important to RAG quality than swapping one strong generator for another. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Store the raw file, page image, extracted text, coordinates, confidence, and parser version. Support visual verification when confidence is low. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[IBM-DOCLING] IBM Granite Docling

<https://www.ibm.com/granite/docs/models/docling> • accessed 2026-07-10

14. Perplexity

Sonar APIs specialize in web-grounded answers, reasoning, and deep research with citations. They can simplify online research integrations. Pricing includes search/request or research components, and citation quality still requires independent validation.

Models covered

- Sonar (sonar) — Current search-grounded API model; search; Text with web search grounding.
- Sonar Pro (sonar-pro) — Current advanced search-grounded API model; search; Text with web search grounding.
- Sonar Reasoning Pro (sonar-reasoning-pro) — Current reasoning plus search model; reasoning; Text with web search grounding.
- Sonar Deep Research (sonar-deep-research) — Current research-specialized API model; search; Text with iterative web research.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Sonar

Model ID / alias: sonar • **Status:** Current search-grounded API model • **Archetype:** search

Tags: search, citations, web-grounded, current-information, api

Access	Perplexity API
Modalities	Text with web search grounding
Advertised context	See live API model card
Advertised max output	Text with citations/search results
Published pricing snapshot	\$1/MTok input and \$1/MTok output, plus search-request fees that vary by search context; verify current pricing

What it is and why it matters

Perplexity positions Sonar as its economical, search-grounded online model. Combines generation with live search and citations for time-sensitive questions, market scans, current events, and web research.

Core capabilities

- Answers questions using live web search and returns citations.
- Optimized for lightweight grounded question answering rather than fully autonomous deep research.
- Combines model-token charges with search-request pricing.

Best-fit use cases

- Current-events Q&A, web-grounded assistants, market monitoring, and research discovery.
- Applications that need citations without building a full search stack.
- Fast factual lookups with a controlled web-search budget.

When not to choose it first

- Private-corpus-only workflows where public web search is unnecessary or disallowed.
- Treating citations as proof that every claim is entailed by the cited page.
- Workloads with unpredictable search depth and no cost controls.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated search capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Search grounding inherits web quality problems: SEO spam, duplicates, inaccessible primary sources, changing pages, and citation-to-claim mismatch. Request fees can dominate token cost. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Evaluate citation entailment, source quality, freshness, and coverage separately. Search fees can dominate token fees for short answers. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Treat web search as a separate retrieval source with a trust policy. Prefer primary domains, store retrieval timestamps, and distinguish public web evidence from private documents. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[PPLX-MODELS] Perplexity Sonar models

<https://docs.perplexity.ai/docs/sonar/models> • accessed 2026-07-10

[PPLX-PRICE] Perplexity pricing

<https://docs.perplexity.ai/docs/getting-started/pricing> • accessed 2026-07-10

Sonar Pro

Model ID / alias: sonar-pro • **Status:** Current advanced search-grounded API model • **Archetype:** search

Tags: search, research, citations, web-grounded, api

Access	Perplexity API
Modalities	Text with web search grounding
Advertised context	See live API model card
Advertised max output	Longer grounded answers with citations
Published pricing snapshot	\$3/MTok input and \$15/MTok output, plus applicable request/search fees; verify current pricing

What it is and why it matters

Perplexity positions Sonar Pro as a higher-quality online search model. Combines generation with live search and citations for time-sensitive questions, market scans, current events, and web research.

Core capabilities

- More capable search-grounded generation than the base Sonar tier.
- Designed for richer answers, more complex queries, and stronger synthesis across web sources.
- Returns citations and can be integrated as a research component in an agent.

Best-fit use cases

- Market research, competitive intelligence, current technical research, and source-backed content drafting.
- Applications where answer quality matters more than the lowest possible web-Q&A cost.
- Research assistants that log and validate source provenance.

When not to choose it first

- High-volume trivial queries that the base tier or a cached retrieval system can answer.
- High-stakes research without independent source validation.
- Private data tasks where web search adds no value.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated search capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Search grounding inherits web quality problems: SEO spam, duplicates, inaccessible primary sources, changing pages, and citation-to-claim mismatch. Request fees can dominate token cost. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Do not confuse cited synthesis with exhaustive research. Important claims should be checked against primary sources. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Treat web search as a separate retrieval source with a trust policy. Prefer primary domains, store retrieval timestamps, and distinguish public web evidence from private documents. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[PPLX-MODELS] Perplexity Sonar models

<https://docs.perplexity.ai/docs/sonar/models> • accessed 2026-07-10

[PPLX-PRICE] Perplexity pricing

<https://docs.perplexity.ai/docs/getting-started/pricing> • accessed 2026-07-10

Sonar Reasoning Pro

Model ID / alias: **sonar-reasoning-pro** • Status: **Current reasoning plus search model** • Archetype: **reasoning**

Tags: reasoning, search, citations, agents, current-information

Access	Perplexity API
Modalities	Text with web search grounding
Advertised context	See live API model card
Advertised max output	Reasoned text with citations
Published pricing snapshot	\$2/MTok input and \$8/MTok output plus applicable search/request fees; verify current pricing

What it is and why it matters

Perplexity offers a reasoning-oriented Sonar tier for more complex online questions. Optimized for deliberate multi-step reasoning, planning, coding, mathematics, and hard analytical tasks where extra inference work can improve reliability.

Core capabilities

- Combines explicit reasoning behavior with live web retrieval.
- Useful for questions that require both source discovery and multi-step synthesis.
- Can act as a specialized research step in a larger agent pipeline.

Best-fit use cases

- Complex current-information questions, comparative analysis, and source-grounded planning.
- Agent workflows that need a web-research specialist.
- Problems where reasoning length must be budgeted separately from final answer length.

When not to choose it first

- Simple lookups where reasoning increases cost and latency.
- Blind trust in intermediate reasoning or citation correctness.
- Tasks where the relevant evidence is entirely inside a private knowledge base.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated reasoning capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Reasoning tokens increase cost and latency and can still produce a confident wrong path. Internal reasoning is not auditable evidence. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Reasoning can amplify an incorrect premise or weak source. Validate the retrieved evidence before scoring the final reasoning. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Use for evidence reconciliation, retrieval planning, and analytical answers. Require source-linked conclusions and use lower-cost models for easy requests. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[PPLX-MODELS] Perplexity Sonar models

<https://docs.perplexity.ai/docs/sonar/models> • accessed 2026-07-10

[PPLX-PRICE] Perplexity pricing

<https://docs.perplexity.ai/docs/getting-started/pricing> • accessed 2026-07-10

Sonar Deep Research

Model ID / alias: sonar-deep-research • **Status:** Current research-specialized API model • **Archetype:** search

Tags: deep-research, search, citations, reports, web

Access	Perplexity API
Modalities	Text with iterative web research
Advertised context	See live API model card
Advertised max output	Research reports with citations
Published pricing snapshot	\$2/MTok input and \$8/MTok output plus citation, search, and reasoning charges; verify current pricing

What it is and why it matters

Perplexity positions Deep Research for comprehensive online investigation. Combines generation with live search and citations for time-sensitive questions, market scans, current events, and web research.

Core capabilities

- Performs iterative research over the web and produces longer source-backed reports.
- Designed for tasks where breadth of source discovery and synthesis are more important than single-call latency.
- Pricing can include multiple metered components beyond ordinary input/output tokens.

Best-fit use cases

- Literature discovery, market scans, policy research, due-diligence support, and technical landscape reports.
- Research steps inside a user-controlled workflow, when supported by the application.
- Generating a source map before human review.

When not to choose it first

- Low-latency chat, simple FAQs, or tightly bounded private-corpus questions.
- Unbounded autonomous research without time, domain, source-quality, and spending limits.
- Presenting the report as authoritative without checking primary sources.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated search capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

Search grounding inherits web quality problems: SEO spam, duplicates, inaccessible primary sources, changing pages, and citation-to-claim mismatch. Request fees can dominate token cost. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Track every retrieved source, query, date, and metered component. Deep research should have explicit stop conditions and a human validation stage. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Treat web search as a separate retrieval source with a trust policy. Prefer primary domains, store retrieval timestamps, and distinguish public web evidence from private documents. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources**[PPLX-MODELS] Perplexity Sonar models**

<https://docs.perplexity.ai/docs/sonar/models> • accessed 2026-07-10

[PPLX-PRICE] Perplexity pricing

<https://docs.perplexity.ai/docs/getting-started/pricing> • accessed 2026-07-10

15. Writer

Palmyra models sit inside an enterprise content, governance, and agent platform. Long-context and competitive token pricing are notable. Buyers should evaluate ecosystem fit, lifecycle notices, exportability, and migration behavior between Palmyra generations.

Models covered

- Palmyra X5 (palmyra-x5) — Current enterprise model; balanced; Text.
- Palmyra X4 / legacy Palmyra models (palmyra-legacy) — Legacy/deprecating family; verify endpoint lifecycle; balanced; Text.

Portfolio operating guidance

Compare at least one high-capability model, one balanced model, and one low-cost or small model on the same internal workload. Treat provider-native and cloud-catalog endpoints as separate products because features, regions, context limits, privacy terms, and prices may differ. Confirm the exact ID, version, preview or GA status, region, data policy, quota, tool support, price units, and retirement policy.

Palmyra X5

Model ID / alias: palmyra-x5 • **Status:** Current enterprise model • **Archetype:** balanced

Tags: enterprise, long-context, agents, business, api

Access	Writer platform/API and supported enterprise deployments
Modalities	Text
Advertised context	Up to 1M tokens according to Writer documentation
Advertised max output	See live model card
Published pricing snapshot	\$0.60/MTok input and \$6/MTok output in published API pricing; verify current tier

What it is and why it matters

Writer positions Palmyra X5 as a long-context enterprise model and lists a one-million-token context window. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- Enterprise generation, long-context document synthesis, agentic workflows, structured extraction, and business writing.
- Designed to integrate with Writer's enterprise application, governance, and knowledge-graph ecosystem.
- Long context can support large document sets or extensive workflow state.

Best-fit use cases

- Enterprise content operations, long-document analysis, business agents, and governed knowledge workflows.
- Organizations already using Writer's application and governance stack.
- Cost-aware long-context API use after realistic output-volume testing.

When not to choose it first

- Assuming a one-million-token window guarantees accurate recall across the whole prompt.
- Relying on a proprietary ecosystem without an exit and export plan.
- Migrating from older Palmyra endpoints without regression tests.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations, instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Output price is materially higher than input price, so verbose agents and report generation need explicit token budgets. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[WRITER-MODELS] Writer Palmyra models

<https://dev.writer.com/home/models> • accessed 2026-07-10

[WRITER-PRICE] Writer API pricing

<https://dev.writer.com/home/pricing> • accessed 2026-07-10

Palmyra X4 / legacy Palmyra models

Model ID / alias: palmyra-legacy • **Status:** Legacy/deprecating family; verify endpoint lifecycle • **Archetype:** balanced

Tags: legacy, migration, deprecation, enterprise, lifecycle

Access	Writer platform/API for supported legacy tenants
Modalities	Text
Advertised context	Model-dependent
Advertised max output	Model-dependent
Published pricing snapshot	Legacy pricing varies

What it is and why it matters

Writer has published lifecycle notices moving customers from older Palmyra models toward newer releases. Provides a practical quality, latency, and cost balance for production chat, summarization, extraction, classification, and moderate agents.

Core capabilities

- Earlier enterprise-language-model generation used for business content and knowledge workflows.
- Relevant primarily for existing deployments and migration planning.
- May have prompt, formatting, and output behavior that differs substantially from X5.

Best-fit use cases

- Temporary support of existing applications during a controlled migration.
- Creating regression datasets that capture old-model behavior.
- Documenting dependency and deprecation risk.

When not to choose it first

- New production systems when a supported replacement is available.
- Last-minute endpoint swaps without prompt and evaluation changes.
- Assuming a newer model is behaviorally drop-in compatible.

Advantages

This model can be assigned a clear role in a routed architecture rather than being treated as a universal endpoint. Its practical advantages are the stated balanced capabilities, its available deployment route, and the possibility of comparing it with cheaper or stronger siblings. Keep prompts, tool schemas, response validation, and telemetry behind an internal provider-neutral interface so the application remains portable.

Practical limitations and failure modes

It may underperform flagship reasoning models on ambiguous multi-step tasks, difficult debugging, advanced mathematics, and long-horizon planning. Too many retrieved chunks can reduce quality. Cross-cutting risks include hallucinated facts or citations,

instruction conflicts, prompt injection, stale knowledge, overconfidence, format drift, prompt sensitivity, and uneven performance across languages and domains. Writer's published lifecycle information indicated removal of several older models around July 13, 2026. Confirm the exact tenant and endpoint status before use. A public benchmark result is not evidence that the model will succeed on your corpus, tools, policies, or latency target.

RAG fit

Usually the best default generator for enterprise RAG when paired with hybrid retrieval, reranking, citations, deterministic prompting, and escalation to a stronger model. Retrieve only authorized evidence, preserve source and page metadata, rerank candidates, ask for source-linked answers, and force an explicit insufficient-evidence outcome when the corpus cannot support a claim. Evaluate indexing, query rewriting, retrieval, reranking, context packing, and generation as separate stages.

Deployment and cost notes

Published list price is only one part of cost. Include cached versus uncached input, output length, reasoning tokens, search or tool fees, audio/image units, batch discounts, retries, vector search, reranking, observability, and engineering operations. For self-hosting, include GPU memory, utilization, redundancy, networking, power, orchestration, support, and upgrade labor.

Evaluation checklist

- Run a representative golden set covering easy, ambiguous, adversarial, multilingual, long-context, and no-answer queries.
- Measure factual correctness, evidence entailment, citation precision, completeness, refusal quality, schema validity, latency, and total cost.
- Test prompt-injection resistance and authorization boundaries with retrieved documents that contain hostile instructions.
- Pin the exact model or revision where possible and rerun the suite before any alias, endpoint, quantization, or prompt-template change.

Official sources

[WRITER-MODELS] Writer Palmyra models

<https://dev.writer.com/home/models> • accessed 2026-07-10

[WRITER-PRICE] Writer API pricing

<https://dev.writer.com/home/pricing> • accessed 2026-07-10

Part III — RAG engineering, evaluation, and operations

Chunking and metadata design

Chunking should follow document structure rather than a universal character count.

Narrative policies and manuals: split on headings, then paragraphs, with modest overlap when references cross boundaries. Tables: retain headers, units, row/column relationships, and a link to the original table or page image. Consider one chunk per logical row group plus a table summary. Code: chunk by file, class, function, or symbol; preserve repository path, commit, language, imports, and call relationships. Contracts and regulations: preserve section numbers, definitions, amendments, effective dates, and cross-references. Slides: keep slide number, title, speaker notes, and visual description. Scans: retain OCR confidence and bounding boxes. Conversations: separate participant, timestamp, thread, and message-level permissions.

Recommended metadata: doc_id, chunk_id, parent_id, source_uri, source_type, title, provider, model_id, section_path, page_start, page_end, language, jurisdiction, effective_from, effective_to, created_at, updated_at, ingested_at, corpus_version, parser_name, parser_version, embedding_model, embedding_dimension, content_hash, tenant_id, owner, confidentiality, acl, pii_flags, tags, and source_authority.

Do not place the only copy of critical metadata inside the embedded text. Use filterable structured fields. Treat document-level and chunk-level metadata separately.

Retrieval, reranking, and context packing

Hybrid retrieval is a strong default because semantic and lexical methods fail differently. Dense vectors find paraphrases and concepts. BM25 or another lexical method finds exact product codes, names, error messages, legal citations, and rare terms. Structured filters enforce tenant, language, date, document type, and authorization. Graph or SQL retrieval is better when relationships or exact aggregation matter.

Retrieve broadly enough to protect recall, then rerank a bounded set. Measure retrieval Recall@k, MRR, nDCG, and coverage on real queries. Reranking should not remove all diversity: multiple independent passages may be necessary to answer a question or reveal a contradiction.

Context packing rules: - Put instructions outside and above retrieved content. - Mark retrieved text as untrusted data, never as instructions. - Keep stable chunk IDs next to every passage. - Prefer fewer complete, authoritative passages to many partial duplicates. - Preserve headings, dates, units, and table context. - Order evidence deliberately; test primacy, recency, and lost-in-the-middle effects. - Reserve space for output and tool calls. - Do not exceed a model's context window merely because the API accepts a large prompt.

Failure modes and mitigations

Hallucination: the model states unsupported facts. Mitigate with evidence-only prompting, no-answer behavior, citation validation, and human review. Retrieval miss: the right chunk is absent. Improve parsing, metadata, query decomposition, hybrid search, synonyms, and corpus coverage. Retrieval noise: distractors dominate. Use reranking, deduplication, authority weighting, and smaller context. Citation mismatch: a cited source does not support the sentence. Validate claim-to-passage entailment and show quoted evidence snippets where appropriate. Prompt injection: a document tells the model to ignore policy or call a tool. Treat documents as untrusted, isolate instructions, restrict tools, and scan content. Authorization leak: retrieval returns another user's data. Enforce ACL filters in the retrieval layer; never rely on the model to hide it. Stale answer: an older document outranks the effective version. Model effective dates and authority, remove superseded chunks, and display freshness. Long-context degradation: evidence is accepted but ignored. Retrieve selectively, summarize hierarchically, and test evidence placement. Multilingual drift: meaning or policy changes across languages. Evaluate per language and preserve original passages. Numeric error: totals, dates, or units are misread. Use deterministic calculators, SQL, schema checks, and source-value comparison. Tool error: the model generates invalid or dangerous arguments. Validate schemas and permissions, use idempotency, confirmations, and allowlists. Format drift: output violates JSON or a template. Use structured outputs, parsers, retries, and deterministic post-validation. OCR/layout error: text order or table relationships are corrupted. Preserve images and coordinates and route low-confidence cases for visual review. Model update regression: an alias changes behavior. Pin versions where possible and gate every change through regression tests.

Evaluation framework

Build an evaluation set from real user tasks, historical failures, domain-expert examples, and adversarial tests. Separate answerable and unanswerable questions. Label the required evidence and acceptable answer variants.

Retrieval metrics: - Recall@k: whether required evidence appears in the candidate set. - MRR and nDCG: ranking quality. - ACL precision: whether unauthorized chunks are always excluded. - freshness/authority accuracy: whether current and authoritative documents are preferred.

Generation metrics: - correctness and completeness, - faithfulness to evidence, - citation precision and recall, - no-answer precision and recall, - instruction and schema compliance, - tone and domain terminology, - multilingual quality, - tool-call validity, - safety and privacy outcomes.

Operational metrics: - end-to-end p50/p95/p99 latency, - time to first token, - successful requests per dollar, - input, cached input, reasoning, and output tokens, - retrieval and reranking latency, - retry and timeout rate, - human escalation rate, - user correction and abandonment rate.

Use pairwise human review for meaningful model comparisons. Blind reviewers to provider names. Define materiality: a cheaper model is not better if it creates more escalations or incorrect business actions.

Security, privacy, and governance

A RAG system expands the attack surface because it connects models, proprietary documents, identity systems, vector databases, external tools, and logs.

Core controls: - identity-aware retrieval with deny-by-default filters, - encryption in transit and at rest, - tenant isolation and document-level ACLs, - secrets management and short-lived credentials, - data-loss prevention for prompts, outputs, and logs, - explicit retention and deletion propagation into indexes, - provider data-use and residency review, - tool allowlists, parameter validation, and least privilege, - prompt-injection and exfiltration testing, - moderation and abuse monitoring, - immutable audit records for sensitive actions, - human approval for consequential decisions.

Do not log raw sensitive prompts by default. Redact or tokenize PII, credentials, health data, and financial identifiers. Keep a data map showing which components receive raw documents, extracted text, vectors, prompts, and generated outputs.

Governance records should include model and prompt versions, model card, intended use, prohibited use, evaluation results, risk owner, data sources, licenses, region, vendor terms, incident process, retirement plan, and approval status.

Agents and tool use

An LLM agent is a policy-driven loop around a model, tools, memory, and state. The model proposes actions; the application decides whether they are valid and permitted.

Design principles: - Prefer deterministic workflows when the steps are known. - Give each tool a narrow schema, clear description, bounded output, and least-privilege credential. - Separate read tools from write tools. - Validate every argument outside the model. - Require confirmation for irreversible, expensive, external, or sensitive actions. - Use idempotency keys and transaction boundaries. - Set maximum steps, time, token, and spend budgets. - Detect repeated plans and stop loops. - Keep retrieved text and tool output marked as untrusted. - Recheck authorization at execution time. - Summarize state rather than allowing an unbounded conversation transcript. - Record action, evidence, model, tool result, and final disposition.

A stronger model may improve planning but does not replace workflow controls. Agent reliability is a systems-engineering property.

Multimodal, realtime voice, and document AI

Multimodal systems should keep modality-specific evidence instead of converting everything into an untraceable paragraph.

For images and pages: - preserve the original image, resolution, page number, crop coordinates, and OCR output; - use a document parser for reading order and tables; - use a vision model when visual relationships matter; - test small text, rotated scans, charts, legends, and handwriting; - display the source page for human verification.

For voice: - store a timestamped transcript subject to consent and retention rules; - confirm names, numbers, dates, addresses, and high-risk instructions; - handle interruptions, silence, noise, accents, and crosstalk; - measure first-audio latency and turn-end detection; - retrieve textual evidence before speaking the answer; - provide citations in a companion visual interface where possible.

For video: - sample frames and align them with transcript timestamps; - avoid assuming a model observed every frame; - preserve temporal provenance for claims.

Deployment patterns and model routing

Managed API: Best for rapid delivery, elastic scale, and access to frontier capability. Risks include variable price, vendor dependency, data-policy constraints, regional availability, quotas, and model lifecycle.

Cloud model catalog: Useful for consolidated identity, networking, governance, procurement, and multiple providers. A catalog endpoint may expose a different feature set or release schedule from the model vendor's native API.

Self-hosted open weights: Best for control, data locality, tuning, and sustained-volume economics when utilization is high. The organization owns capacity planning, inference optimization, safety, upgrades, monitoring, and license compliance.

Edge/on-device: Useful for privacy, offline use, and very low network latency. Model size, battery, memory, context, and update distribution are constraints.

Routing: Use rules or a learned router based on risk, complexity, modality, language, context size, latency objective, and budget. Keep a safe fallback. Evaluate the router itself: wrong routing can erase savings or reduce quality. A common cascade is small utility model → balanced generator → reasoning/frontier escalation → human review.

Lifecycle and migration playbook

Model retirement is a normal operating condition.

Maintain an inventory of every model ID, alias, region, API version, prompt, tool schema, embedding index, evaluation result, and owner. Subscribe to provider lifecycle notices. Avoid undocumented aliases in production. Set internal support windows earlier than the vendor's final date.

Migration steps: 1. Select a candidate replacement and confirm feature parity. 2. Run offline retrieval and generation regression tests. 3. Recalculate tokenizer usage and cost. 4. Test structured output, function calling, safety, multilingual behavior, context limits, and streaming. 5. Shadow production traffic without affecting users. 6. Canary a small percentage with rollback. 7. Compare human-rated quality and operational metrics. 8. Update prompts rather than assuming drop-in behavior. 9. Re-embed the corpus if the embedding model changes; do not mix incompatible vector spaces. 10. Archive the evaluation and decision record.

Use explicit dates in documentation. "Latest" and "current" are not reproducible identifiers.

Use-case playbooks

Enterprise policy assistant: Use document parsing, hybrid retrieval, reranking, a balanced generator, source-page citations, effective-date filters, and no-answer behavior. Escalate legal or HR interpretations.

Developer assistant: Index code by symbol and repository structure. Include commit hashes and access controls. Use a coding-capable model, tools for search/tests, and write-action approval.

Customer support: Route intent with a fast model, retrieve current product and policy evidence, generate with a balanced model, and escalate sentiment, refund, medical, legal, or security issues. Measure resolution, correctness, and escalation—not only fluency.

Research assistant: Separate private sources from public web search. Prefer primary sources, store retrieval date, validate citations, and use a reasoning or deep-research model only when the complexity justifies it.

Document extraction: Use OCR/layout parsing and schemas before free-form generation. Validate totals and identifiers deterministically. Retain confidence and page coordinates.

Voice agent: Use realtime speech for the interface, but keep a text transcript, identity checks, explicit confirmations, tool limits, and visual evidence for consequential information.

High-stakes domain: Restrict the system to decision support, retrieve only approved evidence, require calibrated uncertainty and human review, and conduct domain-specific safety validation.

TXT ingestion instructions

The TXT artifact is organized as independent records. Each record begins with <<<DOCUMENT>>> and ends with <<<END_DOCUMENT>>>. Metadata appears before ---CONTENT---. This format is designed to be split deterministically before secondary chunking.

Recommended ingestion: 1. Split on <<<DOCUMENT>>>. 2. Parse metadata lines into structured fields. 3. Keep official source URLs as metadata and in the content. 4. For model cards, optionally split on all-caps section headings while retaining provider, model_id, as_of_date, and source_ids. 5. Use hybrid indexing. Embed the prose but also index provider, model name, aliases, tags, modality, archetype, and pricing text lexically. 6. Attach an ACL appropriate for your environment. 7. Add a freshness filter or revalidation workflow for pricing and lifecycle questions. 8. Store this corpus version as 2026-07-10 and do not silently overwrite it with later data.

Sample retrieval questions: - Which lower-cost models can route or classify requests before a frontier model? - Compare managed and open-weight options for a private RAG deployment. - Which components improve document retrieval besides the generator? - What cost elements are missing from simple input/output token pricing? - Which models or endpoints have lifecycle warnings? - What failure modes should be tested for long-context multimodal RAG?

Glossary

ACL: access-control list. Agent: model-driven workflow that can select actions or tools. BM25: lexical ranking method based on term frequency and document statistics. Chunk: retrievable segment of a source document. Context window: maximum token capacity accepted by a model endpoint. Dense retrieval: semantic retrieval using vector embeddings. Embedding: numeric representation used for similarity. Entailment: whether evidence logically supports a claim. Grounding: tying an answer to retrieved or verified evidence. Hybrid search: combination of dense, lexical, structured, or graph retrieval. Inference: running a trained model to produce an output. MoE: mixture of experts; only some sub-networks may activate per token. MRR: mean

reciprocal rank. nDCG: normalized discounted cumulative gain. OCR: optical character recognition. Quantization: representing model weights at lower precision to reduce memory and accelerate inference. RAG: retrieval-augmented generation. Reranker: second-stage model that scores query-document relevance. SLM: small language model. TCO: total cost of ownership. Tool calling: model-generated structured request to application code. Vector database: system that indexes and searches embeddings.

Dated pricing and access snapshot

The table below reproduces the concise pricing or cost note stored on each model card as of 2026-07-10. It is intended for discovery, not procurement. A row that says platform-dependent or self-hosted does not mean free: infrastructure and operations may dominate total cost. Revalidate all monetary figures, regions, tiers, and billing units at the official source before use.

Provider	Model / family	Published price or cost note
OpenAI	GPT-5.6 Sol	\$5 input / \$30 output per 1M tokens
OpenAI	GPT-5.6 Terra	\$2.50 input / \$15 output per 1M tokens
OpenAI	GPT-5.6 Luna	\$1 input / \$6 output per 1M tokens
OpenAI	GPT-Realtime 2.1	Audio/text usage pricing; verify live pricing
OpenAI	OpenAI Embeddings	Model dependent; verify live pricing
Anthropic	Claude Fable 5	\$10 input / \$50 output per 1M tokens
Anthropic	Claude Opus 4.8	\$5 input / \$25 output per 1M tokens
Anthropic	Claude Sonnet 5	\$2/\$10 introductory through 2026-08-31; \$3/\$15 from 2026-09-01
Anthropic	Claude Haiku 4.5	\$1 input / \$5 output per 1M tokens
Google	Gemini 3.5 Flash	\$1.50 input / \$9 output; batch \$0.75/\$4.50 per 1M tokens
Google	Gemini 3.1 Pro Preview	\$2/\$12 for prompts <=200K; \$4/\$18 above 200K
Google	Gemini 3.1 Flash-Lite	\$0.25 input (\$0.50 audio) / \$1.50 output; batch \$0.125/\$0.75
Google	Gemini 2.5 Pro	See current pricing page; tiered pricing may apply
Google	Gemini 3.1 Flash Live Preview	\$0.75 text input; \$3 audio input; \$4.50 text output; \$12 audio output per 1M tokens
Amazon	Amazon Nova 2 Lite	Region and service-tier dependent
Amazon	Amazon Nova 2 Sonic	Region and usage dependent
Amazon	Amazon Nova Premier	Region dependent
Amazon	Amazon Nova Micro	Region dependent
Meta	Llama 4 Maverick	No universal API price; infrastructure or partner price applies
Meta	Llama 4 Scout	Self-hosting or partner dependent
Meta	Llama 4 Behemoth	Not standardized
Meta	Llama 3.3 70B Instruct	Infrastructure or partner dependent
Meta	Llama 3.2 1B/3B	Device or infrastructure dependent
Meta	Llama Guard 4	Infrastructure or partner dependent
Mistral AI	Mistral Medium 3.5	Endpoint dependent
Mistral AI	Mistral Small 4	Provider dependent
Mistral AI	Mistral Large 3	Example Bedrock US price about \$0.50 input / \$1.50 output per 1M tokens
Mistral AI	Devstral 2	Example Bedrock US price about \$0.40 input / \$2 output per 1M tokens
Mistral AI	Mistral OCR 4	Page or document pricing; verify live endpoint
Mistral AI	Mistral Moderation 2	Verify current pricing
Cohere	Command A+	Free within launch rate limits; production terms can differ

Provider	Model / family	Published price or cost note
Cohere	Command A	\$2.50 input / \$10 output per 1M tokens
Cohere	Command A Reasoning	Verify live pricing
Cohere	Embed v4.0	Verify current Cohere pricing
Cohere	Rerank v4 Pro	Request or token dependent; verify live pricing
xAI	Grok 4.5	\$2 input / \$6 output per 1M tokens
xAI	Grok 4.3	\$1.25 input / \$2.50 output per 1M tokens
xAI	xAI Voice Agent	\$3 per hour for listed voice agent; speech services priced separately
DeepSeek	DeepSeek V4 Pro	\$0.003625 cache-hit input / \$0.435 cache-miss input / \$0.87 output per 1M tokens
DeepSeek	DeepSeek V4 Flash	\$0.0028 cache-hit input / \$0.14 cache-miss input / \$0.28 output per 1M tokens
Alibaba / Qwen	Qwen 3.7 Max	\$2.50 input / \$7.50 output per 1M tokens at listed 1M tier; tiering and cache can change cost
Alibaba / Qwen	Qwen 3.7 Plus	Tiered; verify current Model Studio pricing
Alibaba / Qwen	Qwen 3.6 Flash	Verify tiered pricing
Alibaba / Qwen	Qwen3-Coder	Self-hosting or endpoint dependent
Alibaba / Qwen	Qwen3 Embedding	Self-hosting or hosted pricing
Alibaba / Qwen	Qwen3 Reranker	Deployment dependent
Microsoft	Phi-4	Deployment and region dependent
Microsoft	Phi-4-mini-instruct	Deployment dependent
Microsoft	Phi-4-reasoning	Deployment dependent
Microsoft	Phi-4-multimodal-instruct	Deployment dependent
Microsoft	Microsoft Model Router	Routing and underlying billing depend on offering
NVIDIA	Nemotron 3 Ultra 550B-A55B	Self-hosting infrastructure cost or partner-platform metering; no single universal price
NVIDIA	Nemotron 3 Super	Example managed-platform rates vary; self-hosted cost depends on GPU utilization
NVIDIA	Nemotron 3 Nano	Low managed-platform token rates are available on some clouds; self-hosted TCO depends on hardware
NVIDIA	Nemotron Nano VL	Self-hosted or partner-platform pricing
IBM	Granite 4.1	Platform or infrastructure dependent
IBM	Granite 4.0	Platform or infrastructure dependent
IBM	Granite Vision	Platform or infrastructure dependent
IBM	Granite Embedding	Platform or infrastructure dependent
IBM	Docling	Open-source software; infrastructure and managed-platform costs apply
Perplexity	Sonar	\$1/MTok input and \$1/MTok output, plus search-request fees that vary by search context; verify current pricing
Perplexity	Sonar Pro	\$3/MTok input and \$15/MTok output, plus applicable request/search fees; verify current pricing
Perplexity	Sonar Reasoning Pro	\$2/MTok input and \$8/MTok output plus applicable search/request fees; verify current pricing
Perplexity	Sonar Deep Research	\$2/MTok input and \$8/MTok output plus citation, search, and reasoning charges; verify current pricing

Provider	Model / family	Published price or cost note
Writer	Palmyra X5	\$0.60/MTok input and \$6/MTok output in published API pricing; verify current tier
Writer	Palmyra X4 / legacy Palmyra models	Legacy pricing varies

Part IV — Official source registry

The following first-party links are the primary research trail for this corpus. They were accessed on 2026-07-10. Some providers render dynamic pricing or region tables; archive an internal copy of material procurement facts and attach the retrieval date to every decision.

[OAI-MODELS] OpenAI model catalog

<https://developers.openai.com/api/docs/models>

Accessed: 2026-07-10

[OAI-PRICING] OpenAI API pricing

<https://developers.openai.com/api/docs/pricing>

Accessed: 2026-07-10

[ANT-MODELS] Anthropic Claude models overview

<https://platform.claude.com/docs/en/about-claude/models/overview>

Accessed: 2026-07-10

[ANT-PRICING] Anthropic Claude pricing

<https://platform.claude.com/docs/en/about-claude/pricing>

Accessed: 2026-07-10

[ANT-DEP] Anthropic model deprecations

<https://platform.claude.com/docs/en/about-claude/model-deprecations>

Accessed: 2026-07-10

[GOO-PRICING] Google Gemini Developer API pricing and model list

<https://ai.google.dev/gemini-api/docs/pricing>

Accessed: 2026-07-10

[GOO-31PRO] Gemini 3.1 Pro Preview

<https://ai.google.dev/gemini-api/docs/models/gemini-3.1-pro-preview>

Accessed: 2026-07-10

[GOO-35FLASH] Gemini 3.5 Flash

<https://ai.google.dev/gemini-api/docs/models/gemini-3.5-flash>

Accessed: 2026-07-10

[GOO-25PRO] Gemini 2.5 Pro

<https://ai.google.dev/gemini-api/docs/models/gemini-2.5-pro>

Accessed: 2026-07-10

[AWS-BEDROCK] Amazon Bedrock pricing

<https://aws.amazon.com/bedrock/pricing/>

Accessed: 2026-07-10

[AWS-MODELS] Amazon Bedrock supported foundation models

<https://docs.aws.amazon.com/bedrock/latest/userguide/models.html>

Accessed: 2026-07-10

[AWS-NOVA2L] Amazon Nova 2 Lite model card

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-2-lite.html>

Accessed: 2026-07-10

[AWS-NOVA2S] Amazon Nova 2 Sonic model card

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-2-sonic.html>

Accessed: 2026-07-10

[AWS-NOVAP] Amazon Nova Premier model card

<https://docs.aws.amazon.com/bedrock/latest/userguide/model-card-amazon-nova-premier.html>

Accessed: 2026-07-10

[META-L4] Meta Llama 4 announcement

<https://ai.meta.com/blog/llama-4-multimodal-intelligence/>

Accessed: 2026-07-10

[META-L4CARD] Meta Llama 4 model card

<https://www.llama.com/docs/model-cards-and-prompt-formats/llama4/>

Accessed: 2026-07-10

[META-L32] Meta Llama 3.2 announcement

<https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>

Accessed: 2026-07-10

[META-GUARD4] Meta Llama Guard 4 model card

<https://www.llama.com/docs/model-cards-and-prompt-formats/llama-guard-4/>

Accessed: 2026-07-10

[META-COOKBOOK] Meta Llama Cookbook

<https://github.com/meta-llama/llama-cookbook>

Accessed: 2026-07-10

[MISTRAL-MODELS] Mistral models overview and lifecycle<https://docs.mistral.ai/models/overview>

Accessed: 2026-07-10

[COHERE-MODELS] Cohere model catalog<https://docs.cohere.com/docs/models>

Accessed: 2026-07-10

[COHERE-APLUS] Cohere Command A+<https://docs.cohere.com/docs/command-a-plus>

Accessed: 2026-07-10

[COHERE-A] Cohere Command A<https://docs.cohere.com/docs/command-a>

Accessed: 2026-07-10

[XAI-MODELS] xAI model catalog<https://docs.x.ai/developers/models>

Accessed: 2026-07-10

[XAI-PRICING] xAI API pricing<https://docs.x.ai/developers/pricing>

Accessed: 2026-07-10

[DEEPSEEK-PRICE] DeepSeek API pricinghttps://api-docs.deepseek.com/quick_start/pricing/

Accessed: 2026-07-10

[DEEPSEEK-V4] DeepSeek V4 release<https://api-docs.deepseek.com/news/news260424>

Accessed: 2026-07-10

[QWEN-MODELS] Alibaba Cloud Model Studio model catalog<https://www.alibabacloud.com/help/en/model-studio/models>

Accessed: 2026-07-10

[QWEN-PRICE] Alibaba Cloud Model Studio pricing<https://www.alibabacloud.com/help/en/model-studio/model-pricing>

Accessed: 2026-07-10

[QWEN3] Qwen3 open-model announcement<https://qwenlm.github.io/blog/qwen3/>

Accessed: 2026-07-10

[QWEN-CODER] Qwen3-Coder announcement<https://qwenlm.github.io/blog/qwen3-coder/>

Accessed: 2026-07-10

[QWEN-EMBED] Qwen3 Embedding and Reranker<https://qwenlm.github.io/blog/qwen3-embedding/>

Accessed: 2026-07-10

[MS-DIRECT] Microsoft Foundry models sold directly by Azure<https://learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/models-sold-directly-by-azure>

Accessed: 2026-07-10

[MS-PARTNER] Microsoft Foundry partner/community models<https://learn.microsoft.com/en-us/azure/foundry/foundry-models/concepts/models-from-partners>

Accessed: 2026-07-10

[MS-RETIRE] Microsoft Foundry model retirement schedule<https://learn.microsoft.com/en-us/azure/foundry/openai/concepts/model-retirement-schedule>

Accessed: 2026-07-10

[MS-REGIONS] Microsoft Foundry model region availability<https://learn.microsoft.com/en-us/azure/foundry-classic/how-to/deploy-models-serverless-availability>

Accessed: 2026-07-10

[NVIDIA-ULTRA] NVIDIA Nemotron 3 Ultra model card<https://build.nvidia.com/nvidia/nemotron-3-ultra-550b-a55b/modelcard>

Accessed: 2026-07-10

[NVIDIA-SUPER] NVIDIA Nemotron 3 Super<https://build.nvidia.com/nvidia/nemotron-3-super-120b-a12b>

Accessed: 2026-07-10

[NVIDIA-VL] NVIDIA Nemotron Nano 12B v2 VL<https://build.nvidia.com/nvidia/nemotron-nano-12b-v2-vl/modelcard>

Accessed: 2026-07-10

[IBM-GRANITE] IBM Granite family<https://www.ibm.com/granite/docs/models/granite>

Accessed: 2026-07-10

[IBM-G41] IBM Granite 4.1<https://www.ibm.com/granite/docs/models/granite4-1>

Accessed: 2026-07-10

[IBM-VISION] IBM Granite Vision<https://www.ibm.com/granite/docs/models/vision>

Accessed: 2026-07-10

[IBM-EMBED] IBM Granite Embedding<https://www.ibm.com/granite/docs/models/embedding>

Accessed: 2026-07-10

[IBM-DOCLING] IBM Granite Docling<https://www.ibm.com/granite/docs/models/docling>

Accessed: 2026-07-10

[IBM-RAG] IBM Granite RAG guide<https://www.ibm.com/granite/docs/use-cases/langchain-rag>

Accessed: 2026-07-10

[PPLX-MODELS] Perplexity Sonar models<https://docs.perplexity.ai/docs/sonar/models>

Accessed: 2026-07-10

[PPLX-PRICE] Perplexity pricing<https://docs.perplexity.ai/docs/getting-started/pricing>

Accessed: 2026-07-10

[PPLX-QUICK] Perplexity Sonar API<https://docs.perplexity.ai/docs/sonar/quickstart>

Accessed: 2026-07-10

[WRITER-MODELS] Writer Palmyra models<https://dev.writer.com/home/models>

Accessed: 2026-07-10

[WRITER-PRICE] Writer API pricing<https://dev.writer.com/home/pricing>

Accessed: 2026-07-10

End note

A strong RAG system is not created by selecting the most famous model. It is created by combining a well-governed corpus, accurate document parsing, high-recall retrieval, precise reranking, a model appropriate to the task, strict authorization, evidence-linked generation, systematic evaluation, cost observability, and a tested migration path.